

Worked for 8s >

Yes. They are conceptually close—they both implement the **Karpathy-style “LLM-maintained wiki” pattern**—but they have diverged quite a lot in implementation and intended use.

For the kind of system where you want to turn software artifacts, Confluence-style documentation, design knowledge, source code context, business concepts, and operational history into a **semantic knowledge layer for coding agents**, I would currently lean toward **OpenKB as the architectural foundation**, but I think **LLM Wiki has several mechanisms worth borrowing**, especially its graph/review/source-traceability approach.

Bottom line

Area	nashsu/llm_wiki	VectifyAI/OpenKB
Core idea	Compiled persistent wiki	Compiled persistent wiki
Product orientation	Desktop knowledge workspace	Knowledge engine / CLI / API
Architecture cleanliness	More feature-rich, more monolithic	Cleaner, easier to embed
Retrieval	Keyword + optional vectors + graph expansion	PageIndex reasoning/tree retrieval, no vector DB
Knowledge graph	Much stronger	Basic wiki links/entities
Long documents	Conventional extraction + optional MinerU	Strong PageIndex hierarchical retrieval
Human review	Strong	Lint-oriented
Agent integration	MCP + HTTP API + Skill	REST API + Skill
Code-agent backend suitability	Good	Very good
Desktop UX	Excellent relative to OpenKB	Basic web workbench
Source traceability	Strong / explicit	Good

Area	nashsu/llm_wiki	VectifyAI/OpenKB
Knowledge purpose/schema control	Very strong	Strong through AGENTS.md
Enterprise extensibility	Moderate	Better starting point
License	GPL-3.0	Apache-2.0
Current GitHub traction	~16.7k stars	~3.8k stars

The license difference alone may be quite important if this becomes something your organisation modifies or embeds internally: LLM Wiki is GPL-3.0, while OpenKB is Apache-2.0. [GitHub](#) +1

1. They start from essentially the same philosophy

Both reject the simplistic model of:

```
documents
  ↓
chunks
  ↓
embeddings
  ↓
vector DB
  ↓
retrieve every question
```

and instead do something closer to:

```
Raw knowledge
  ↓
LLM compilation
  ↓
persistent structured knowledge
  ↓
concepts / entities / relationships / summaries
  ↓
retrieval + agent reasoning
```

LLM Wiki explicitly describes its architecture as:

```
Raw Sources → Wiki → Schema
```

with `index.md`, `log.md`, YAML metadata and `[[wikilinks]]`. [GitHub](#)

OpenKB similarly turns each source into summaries, concepts, entities and cross-document synthesis rather than leaving documents as isolated chunks. [GitHub](#)

So philosophically:

They are closer to knowledge compilers than ordinary RAG systems.

That's an important distinction.

2. The biggest difference is what each project has become

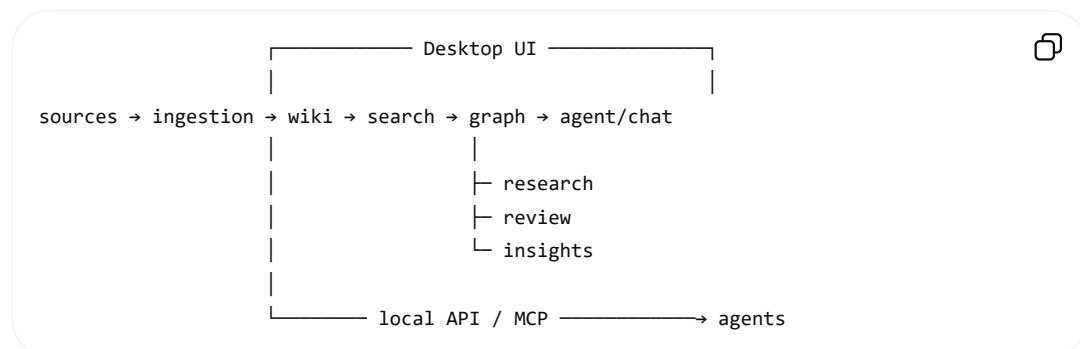
LLM Wiki

LLM Wiki has evolved into something like:

Obsidian + RAG + knowledge graph + research agent + desktop AI workspace

It is a Tauri desktop application with persistent conversations, previews, source browsing, graph analysis, deep research, browser clipping, review workflows and an agent runtime. [GitHub](#) +1

Its structure is therefore roughly:



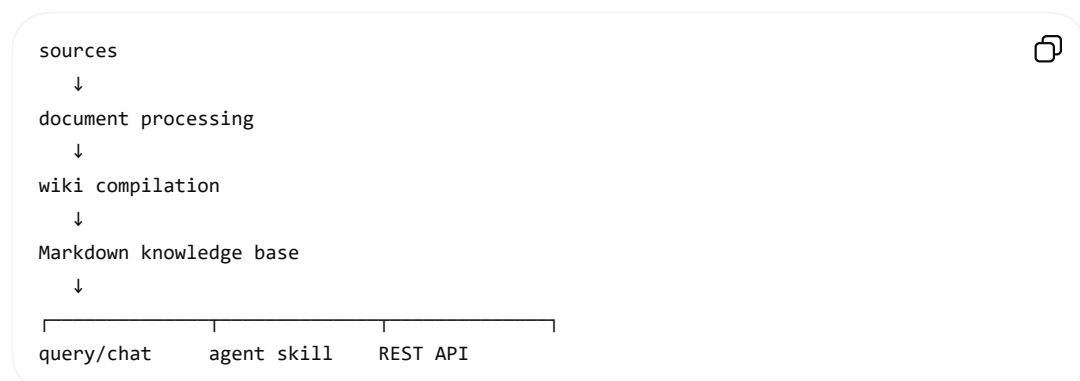
That is a fairly complete **knowledge-work application**.

OpenKB

OpenKB feels more like:

a knowledge compilation and retrieval engine that other agents/apps can consume

Its core looks much more modular:



It has added a web UI, but its conceptual centre remains the knowledge engine rather than the UI. [GitHub](#) +1

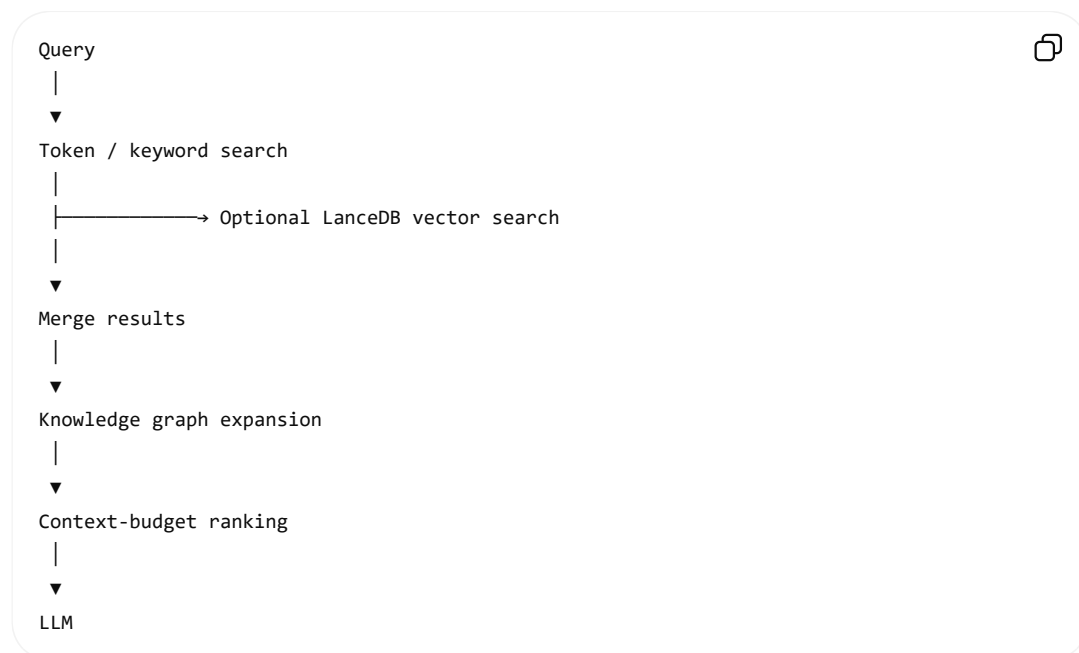
For an enterprise platform architecture, I generally prefer this separation.

3. Retrieval architecture: probably the most important difference

This is where the two projects genuinely take different technical approaches.

LLM Wiki retrieval

LLM Wiki has a multi-stage retrieval pipeline:



Its graph expansion uses four signals:

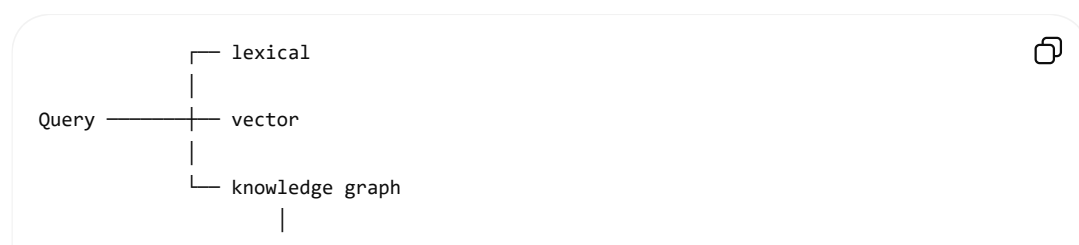
- direct wiki links
- shared source provenance
- Adamic-Adar graph similarity
- type affinity

and does 2-hop graph traversal. [GitHub](#) +1

This is quite sophisticated.

Vector search is optional. Their README reports retrieval recall increasing from **58.2% to 71.4%** when vector retrieval is enabled, although I'd treat a project-published benchmark as indicative rather than independent evidence. [GitHub](#)

Conceptually:



▼
ranked context

That's basically **hybrid retrieval**.

4. OpenKB takes a very different path

OpenKB's most distinctive feature is PageIndex.

For long PDFs:

PDF
↓
PageIndex
↓
hierarchical document tree
↓
reason over tree
↓
identify relevant branches
↓
retrieve content



rather than:

PDF
↓
chunks
↓
embeddings
↓
nearest neighbours



OpenKB calls this **vectorless reasoning-based retrieval**. Documents over its configured threshold—20 pages by default—are represented through a hierarchical tree rather than simply fed wholesale to the model. [GitHub](#)

This is particularly appealing for things such as:

300-page specification
large policy document
architecture standards
business operating manual
regulatory documents



because structure matters enormously.

5. For your use case, however, PageIndex doesn't solve everything

This is important.

Imagine your future knowledge contains:

Confluence

- └ business requirements
- └ architecture decisions
- └ workflow documentation
- └ operating procedures

Azure DevOps

- └ epics
- └ stories
- └ bugs
- └ PR comments
- └ release history

Code

- └ React
- └ Java
- └ SQL
- └ pipeline/config

Figma

- └ screens
- └ interaction rules

Production

- └ logs
- └ observed user behaviour

Your fundamental retrieval problem is not simply:

Find the appropriate section of a large document.

It is:

Find the **relationships between many heterogeneous pieces of information.**

For example:

Business concept

- └ implemented by → Java service
- └ exposed by → REST endpoint
- └ represented by → React component
- └ specified in → Confluence page
- └ designed in → Figma
- └ changed because → Azure DevOps story

And **this is exactly where LLM Wiki's graph machinery is more interesting than OpenKB's current implementation.**

6. LLM Wiki's knowledge graph is significantly richer

LLM Wiki doesn't merely render wiki links.

It computes graph relationships from multiple signals:

```
Direct link
+
Shared source
+
Common neighbours
+
Entity/type affinity
↓
relationship score
```



It then performs Louvain community detection to identify clusters automatically.



It can also identify:

- isolated pages
- sparse knowledge communities
- bridge nodes
- unexpected cross-community connections

and turn those into research/review prompts. 

This is extremely relevant to the idea of a semantic software layer.

Imagine it finding:

```
WorkflowApproval
|
|— SettlementControl
|
|— approval-service
|
|— Story ADO-12883
|
|— ComplianceRule-17
```



and noticing:

```
SettlementControl

has no link to

current UX workflow
```



That potentially becomes:

"Knowledge gap detected."

That's a genuinely interesting capability.

7. Another LLM Wiki idea I particularly like:

purpose.md

LLM Wiki adds something absent from the original Karpathy pattern:

```
purpose.md
```



This defines:

- goals
- scope
- key questions
- evolving thesis

and the LLM reads it during ingestion and query. [GitHub](#)

Think of it as:

```
schema.md
=
HOW knowledge should be represented

purpose.md
=
WHY this knowledge base exists
```



For a software development knowledge base, that might say:

```
purpose:

primary_goal:
  Enable AI coding agents to understand both
  implementation and business intent.

authoritative_sources:
  - architecture decisions
  - approved requirements
  - production code

secondary_sources:
  - Jira/Azure DevOps
  - PR discussions
  - production behaviour

key_questions:
  - Why does this code exist?
  - What business behaviour does it implement?
```

</> YAML



I think this mechanism is extremely useful.

OpenKB's `AGENTS.md` can accomplish some of the same thing—its wiki behaviour and structure are configurable through instructions read at runtime. [GitHub](#)

But conceptually separating:

```
purpose
```



schema

is cleaner.

8. Ingestion quality: LLM Wiki has another nice architectural feature

LLM Wiki performs two logical LLM stages during ingestion.

First:

```
SOURCE
↓
ANALYSIS

entities
concepts
arguments
relationships
contradictions
suggested wiki structure
```



Then:

```
ANALYSIS
↓
GENERATION

summary pages
entity pages
concept pages
links
review items
```



Conceptually, this is much safer than asking:

Read this and immediately modify my knowledge base.

It establishes an intermediate semantic representation:

```
Source
↓
semantic interpretation
↓
knowledge mutation
```



For your system I'd probably go even further:

```
Source
↓
Extraction
↓
Canonical semantic model
```



↓
Validation
↓
Knowledge graph/wiki mutation

which means neither repo is quite the final architecture I would recommend.

9. OpenKB wins convincingly for large-document retrieval

Suppose you ingest a:

600-page operational manual

LLM Wiki's approach is essentially:

parse → wiki/vector index → retrieve

OpenKB gives you:

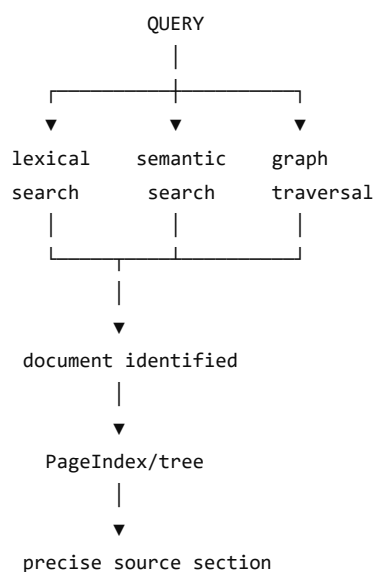
document structure
↓
hierarchical tree
↓
reason about branches
↓
precise retrieval

 GitHub

For regulatory and financial documents, I would strongly favour this.

Therefore I wouldn't actually choose between graph retrieval and PageIndex.

I'd combine them:



That would be much stronger.

10. Source-of-truth handling

LLM Wiki deserves credit here.

Every generated page stores its contributing sources through metadata, and deleting a source triggers cascade cleanup while preserving concepts supported by other sources.  [GitHub](#) +1

Conceptually:

Concept: Four-eye approval 

sources:

- BRD-182
- ADR-004
- workflow_service.java

If:

BRD-182 deleted 

then:

Concept survives 

sources:

- ADR-004
- workflow_service.java

This is closer to **knowledge lineage** than typical RAG.

For finance, I consider that extremely important.

11. Human-in-the-loop: LLM Wiki is stronger

LLM Wiki includes an asynchronous review system where the LLM can flag uncertain items for human judgement.  [GitHub](#)

That enables:

LLM detects: 

Requirement says:

"Manager approval required"

Code indicates:

"Manager OR senior analyst"

↓

REVIEW

Possible contradiction detected.

That's exactly what you want from a knowledge maintenance platform.

OpenKB has knowledge linting and contradiction-oriented compilation, which is useful, but LLM Wiki currently exposes a richer review workflow. [GitHub](#)

12. Agent integration

Both are now quite good, but differently.

LLM Wiki

It exposes:

```
local HTTP API
+
MCP server
+
agent skill
```



Agents can search knowledge, inspect graphs, read files and trigger rescans.

[GitHub](#)

That's very useful with:

```
Claude Code
Codex
VS Code agents
MCP-enabled clients
```



OpenKB

OpenKB exposes a FastAPI REST API with SSE streaming and also supplies a `SKILL.md` that Claude Code, Codex and Gemini can consume directly. [GitHub](#) +1

The interesting philosophy is:

```
Agent
↓
SKILL.md
↓
read compiled Markdown knowledge

rather than necessarily

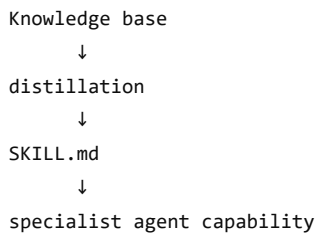
Agent
↓
MCP
↓
running knowledge server
```



That's attractive because it reduces infrastructure dependencies.

13. OpenKB's Skill Factory is particularly interesting for AI SDLC

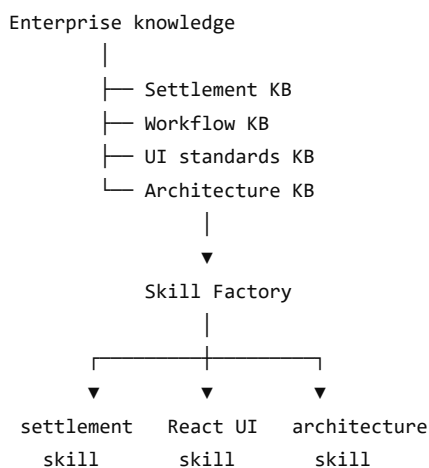
OpenKB can transform a knowledge base into a portable agent skill:



and supports validation, evaluation, history and rollback around generated skills.

GitHub

That suggests a very interesting architecture for a development organisation:



Your coding agent doesn't necessarily need the **entire enterprise KB**.

It loads the relevant distilled capability.

That's a very strong pattern.

14. UI philosophy

Here LLM Wiki wins comfortably.

Its Tauri UI includes:

Knowledge tree
Sources
Chat
Preview
Graph
Lint
Review
Deep Research
Settings

A list of UI features. A copy icon is in the top right corner.

with persistent state and cross-platform desktop builds. GitHub +1

OpenKB now has a Knowledge Workbench served through its web service, but it's still much less central to the project architecture. [GitHub](#)

So for an end user:

LLM Wiki wins.

For a developer building a platform:

OpenKB wins.

15. Technology stack

Very different philosophies again.

LLM Wiki

Mainly:

Tauri
Rust
React/TypeScript
LanceDB
Graphology
Sigma.js
local APIs



It's essentially a native desktop product.

OpenKB

Mainly:

Python
FastAPI
LiteLLM
PageIndex
markdown
Markdown filesystem



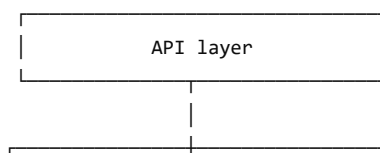
It's closer to an AI backend/service.

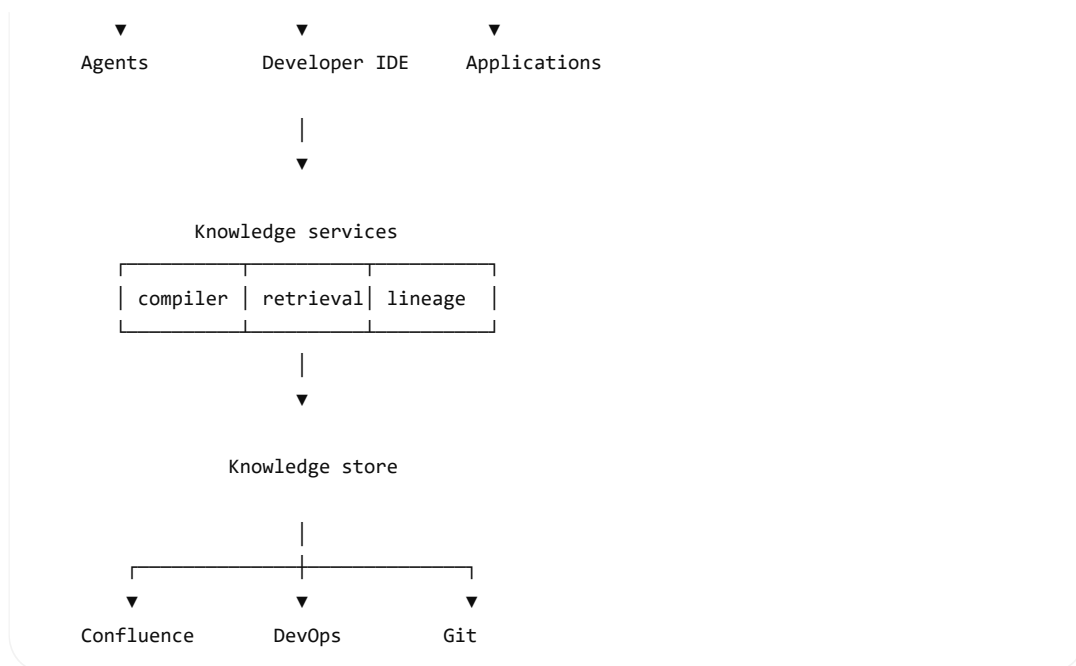
That alone would push me toward OpenKB for an enterprise AI architecture.

16. Enterprise deployment consideration

For something running inside a financial institution, I'd probably want:

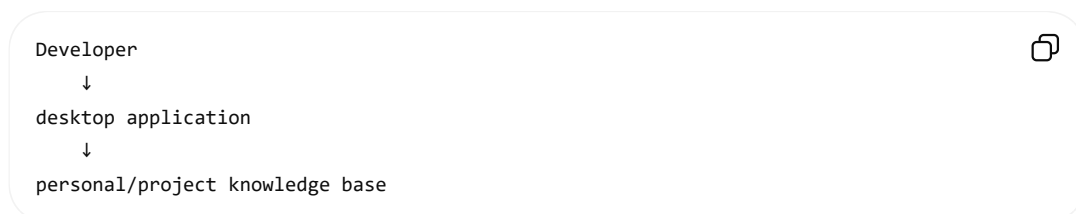
Knowledge Platform





OpenKB fits into the middle of this picture much more naturally.

LLM Wiki feels closer to:



Both are legitimate, but they're different products.

17. There's another very practical issue: licensing

This deserves explicit attention.

LLM Wiki:

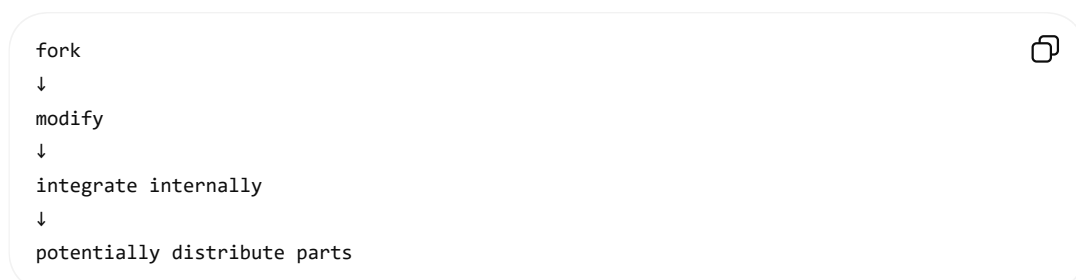
GPL-3.0.  [GitHub](#)

OpenKB:

Apache-2.0.  [GitHub](#)

For experimentation, no big deal.

For:



Apache-2.0 gives you considerably more flexibility.

I'd therefore be much more comfortable using OpenKB as a platform dependency or architectural starting point.

18. Community size currently favours LLM Wiki

As of **24 August 2026**, GitHub shows approximately:

- **LLM Wiki: 16.7k stars / ~2k forks**
- **OpenKB: 3.8k stars / ~412 forks**

 GitHub +1

That doesn't necessarily mean LLM Wiki's underlying architecture is better, but it does suggest substantially more visibility and community experimentation around the desktop project.

19. My scoring for your likely use case

If the objective were simply:

Build a personal AI knowledge manager.

I'd score them approximately:

	LLM Wiki	OpenKB
UX	9	6
Knowledge exploration	9	7
Knowledge graph	9	6
Personal research	9	7
Long docs	7	9
Ease of use	9	7

Winner:

LLM Wiki

But for:

Build an organisational semantic layer behind AI coding agents.

I'd score them differently:

	LLM Wiki	OpenKB
Backend composability	7	9

	LLM Wiki	OpenKB
API orientation	8	9
Agent integration	9	9
Long-document retrieval	7	10
Knowledge graph	9	6
Provenance	9	8
Human review	9	7
Skill generation	7	10
Enterprise licensing	6	10
Architectural simplicity	6	9

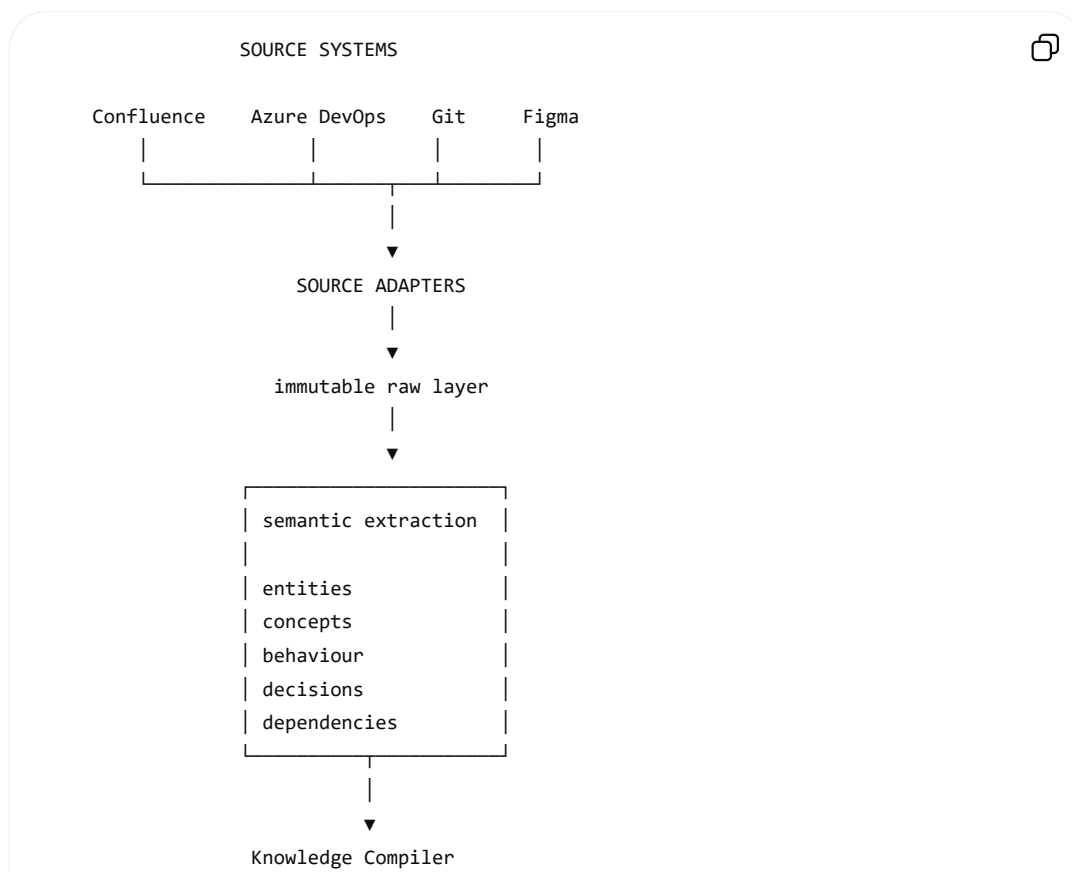
Overall I'd choose:

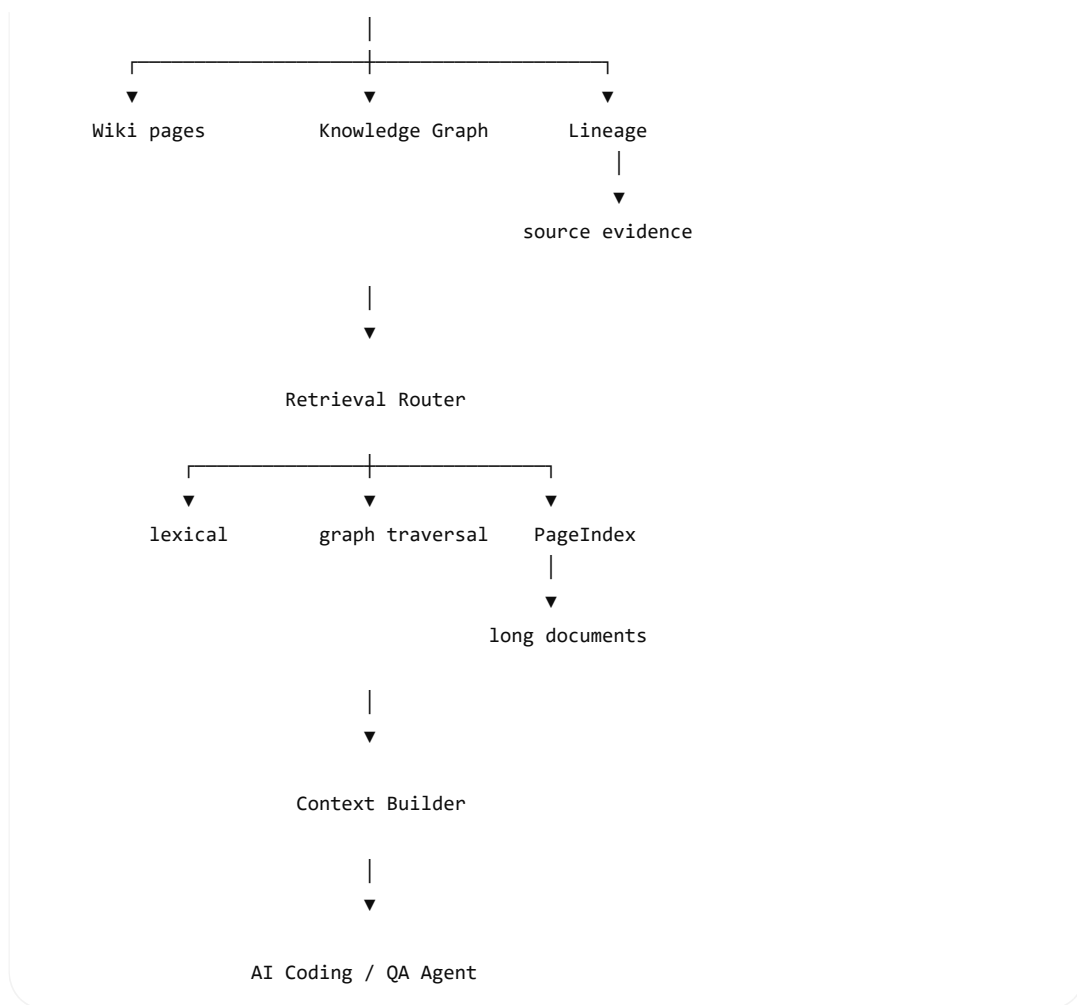
OpenKB as the base architecture.

But I would **not simply deploy vanilla OpenKB**.

20. The architecture I think would be stronger than either repo

I'd combine the best ideas from both.





Then add the **LLM Wiki mechanisms**:

purpose.md
two-stage ingestion
knowledge-gap detection
review queue
community detection
source lineage



and the **OpenKB mechanisms**:

PageIndex
AGENTS.md
REST API
Skill Factory
portable Markdown wiki



That, in my view, is the more interesting direction.

One conceptual change I'd make beyond both repositories

Neither project goes far enough in treating software knowledge as a **typed semantic graph**.

Instead of just:

```
[[Settlement]]
[[Approval]]
[[Workflow]]
```



I'd define relationships such as:

```
Requirement
  IMPLEMENTED_BY → Code

Code
  EXPOSES → API

API
  USED_BY → UI

Story
  MODIFIED → Code

ADR
  JUSTIFIES → Architecture

Test
  VERIFIES → Behaviour

ProductionLog
  EVIDENCES → Behaviour
```



Then your knowledge layer starts becoming:

a software knowledge graph

rather than simply:

an AI-generated wiki.

And that distinction is probably the key to the semantic-layer problem you have been exploring around connecting code with business and development context.

   ...  Sources

Today 1:47 PM

So basically, you recommend to combine both and use their strength in different perspective? After reading your reponse, I want to give more context, from source perspective, I want to add the existing test case repo, which include both system test and regression text etc...currently, every release pipeline related to user story, code, and test we run(this is my understanding, correct me if this is impossible on azure devops). Originally, I wanted to have another grapy knowledge base for code only, to allow human/ai to identify the impact across repos, but it

Show more ▾

Yes — with the extra context, I would refine the recommendation slightly:

Use one unified knowledge platform, but keep different kinds of relationships and evidence explicitly separated. I would *not* build a completely separate code-only graph unless you have a very strong operational reason. The better design is one knowledge model with multiple evidence layers: deterministic code/deployment/test relationships, human-approved business knowledge, and AI-inferred relationships.

And your proposed workflow makes sense. I would actually push it further into a **knowledge reconstruction + continuous assurance workflow**.

Azure DevOps can support much of the traceability you described. It can link requirements/user stories to branches, commits, PRs and builds; Test Plans can link test cases to requirements, and automated test results can be associated with requirements and pipeline runs. That gives you the backbone `User Story → code change → build/release → test → test result`, although the quality of that chain depends on your team actually maintaining the links.  Microsoft Learn +2

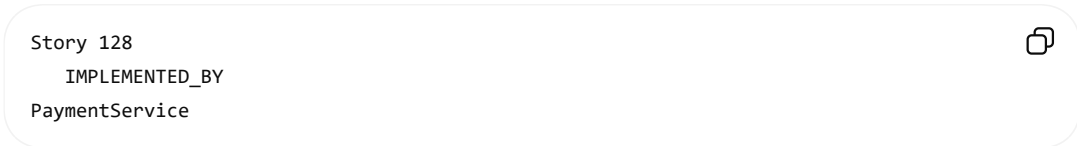
1. I would model this as three knowledge layers

The most important design decision is **do not let AI-extracted relationships look equivalent to facts derived from systems**.

I would explicitly separate:

Relationship type	Example	Reliability
Deterministic	commit <code>abc123</code> belongs to PR 81	Very high
Structural/static-analysis	<code>PaymentController</code> → <code>PaymentService</code> call	High
Operational	test X executed against build 124	Very high
Human-authored	Story 128 implements “four-eye approval”	High, depending on governance
AI-extracted	service appears to implement four-eye approval	Probabilistic
AI-hypothesised	changing approval policy may affect regression test 17	Probabilistic

So the graph should never simply contain:



Instead, conceptually it should contain something like:

Story 128
— IMPLEMENTED_BY → PaymentService

evidence_type: deterministic
derived_from:
- PR #812
- commit 8a182...
confidence: 1.0

versus:

FourEyeApproval
— IMPLEMENTED_BY → ApprovalService

evidence_type: ai_inferred
derived_from:
- ApprovalService.java
- ApprovalValidator.java

confidence: 0.84
model: xxx
extracted_at: ...
review_status: pending

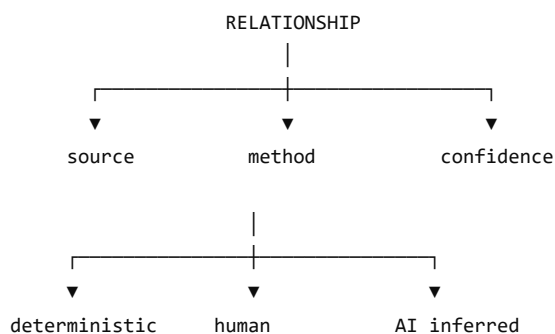
That distinction is **essential** in a financial institution.

2. Think of the knowledge graph as having an “evidence plane”

Instead of just:

Node — relationship — Node

I would model:



For example:

User Story 418
|
| implemented_by
▼
SettlementService
|
| validated_by

Each edge carries provenance.

That gives you a very useful ability:

“Show me only relationships that are objectively established.”

or:

“Show me inferred relationships with confidence > 0.8.”

or:

“Show me unreviewed AI-generated relationships affecting settlement.”

This is much stronger than a conventional wiki.

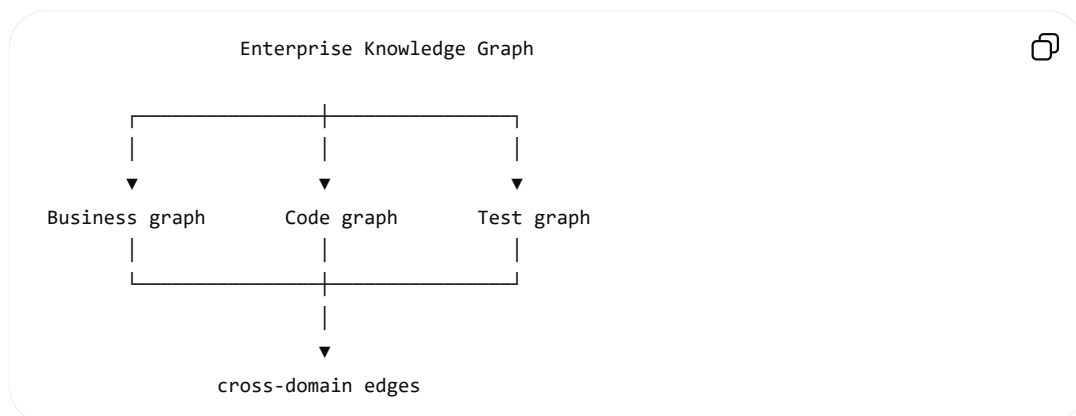
3. Your code-only graph idea is still valid — but I'd make it a *subgraph*, not a separate KB

Originally you wanted:

another graph knowledge base for code only, allowing human/AI to identify impact across repos.

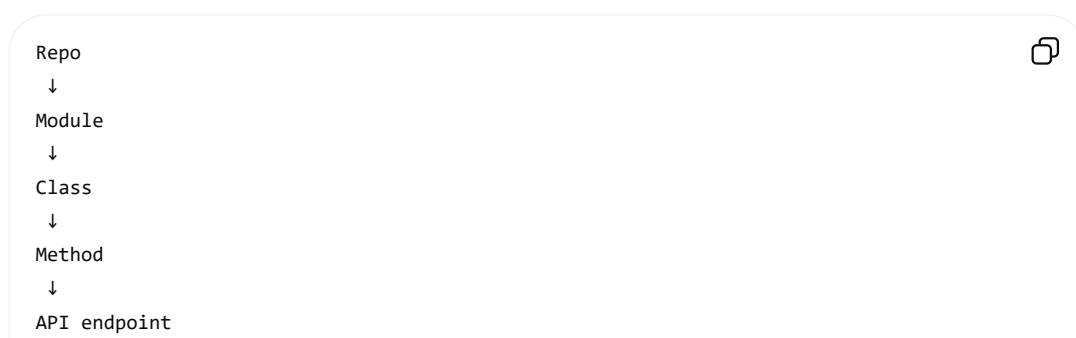
The need is valid.

But I would represent it as:



The **code graph** can still be generated with specialised tooling.

For example:



↓
Database table

with deterministic relationships such as:

```
imports
calls
inherits
implements
reads_from
writes_to
publishes
subscribes_to
depends_on
```



That graph should ideally come from:

- AST parsing
- language server/indexer information
- call graph analysis
- dependency files
- API definitions
- SQL lineage
- build definitions

rather than LLM inference.

Then OpenKB/LLM-Wiki-type processing operates **above** it.

That separation is useful:

LOW LEVEL

code intelligence graph
deterministic / machine-derived

HIGH LEVEL

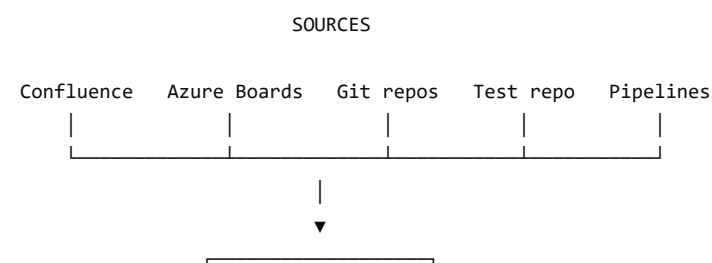
semantic knowledge graph
business concepts / behaviour / requirements

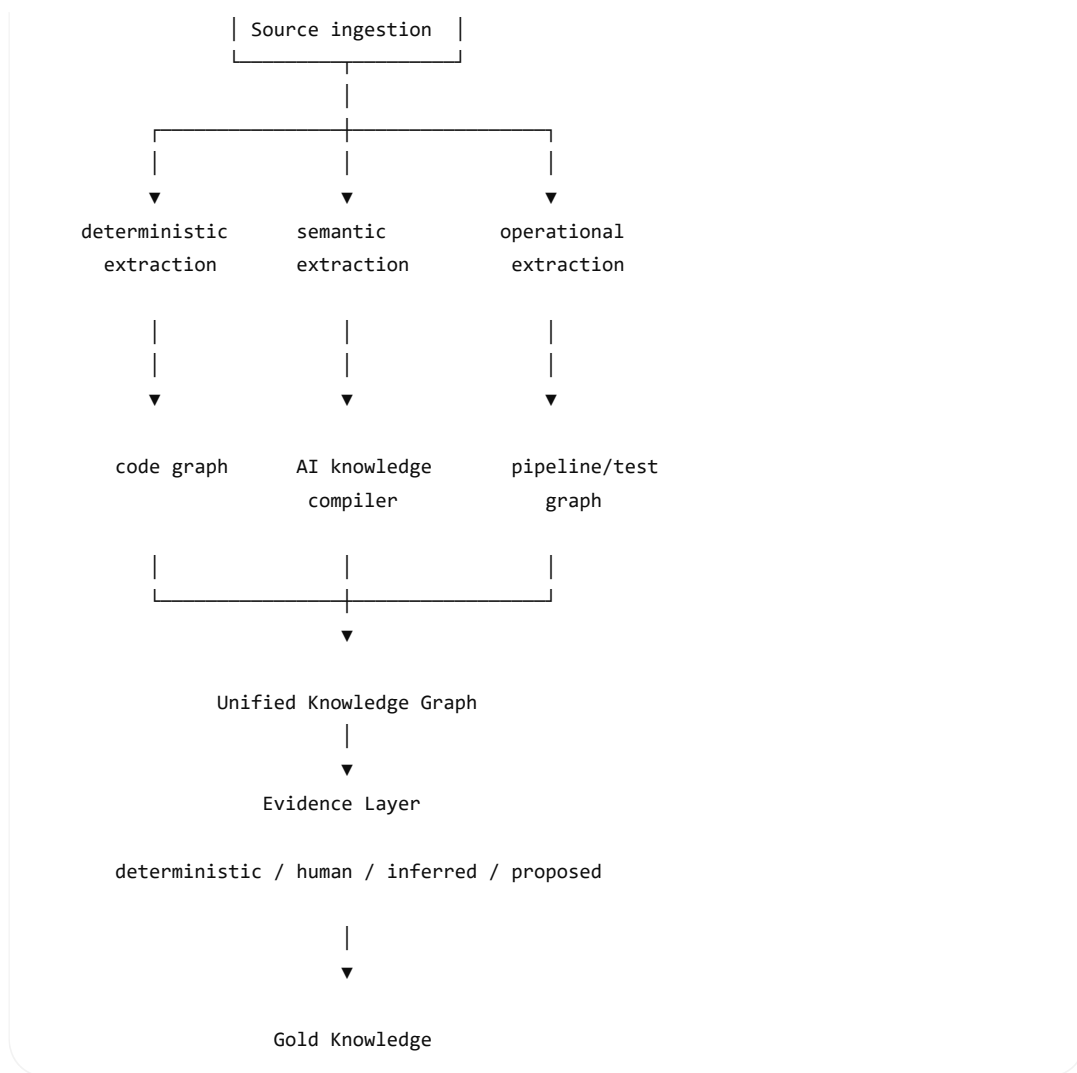


and then you connect them.

4. This is the architecture I'd recommend

Something along these lines:





This is basically the combined idea we discussed, but with a **deterministic graph underneath the LLM wiki**.

That is an important addition.

5. Your test repository becomes extremely valuable

Adding your regression/system test repository is one of the better additions you could make.

Tests often contain **implicit behavioural specifications**.

For example:

```
Given
  payment amount > threshold

When
  user submits settlement

Then
  manager approval must occur
```



Even if nobody ever wrote an acceptance criterion, that test contains one.

Therefore you can reverse engineer:

```
test implementation
  ↓
observed behaviour
  ↓
candidate acceptance criterion
```



Similarly:

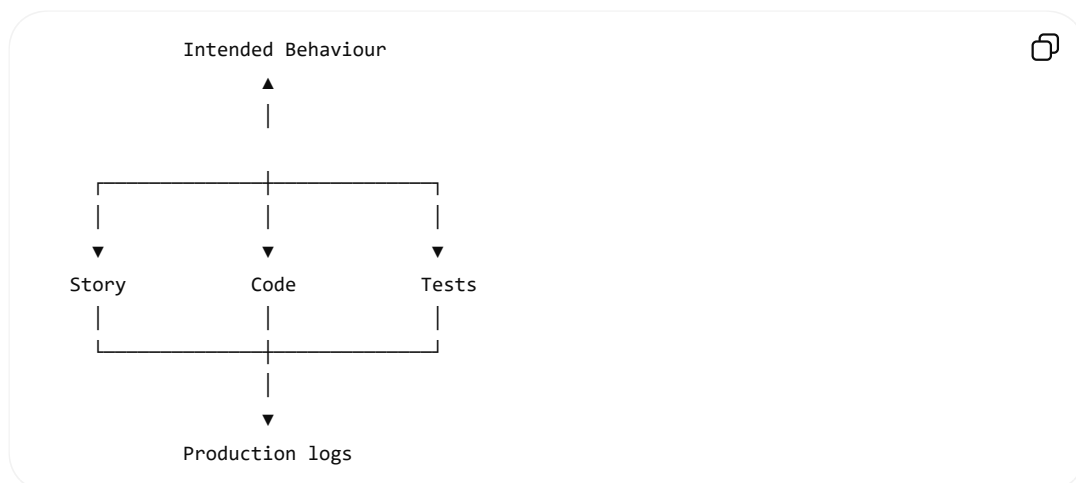
```
code
+
test
+
production behaviour
+
user story
  ↓
reconstructed requirement
```



This is much stronger than reading code alone.

6. I'd treat tests as specification evidence, not just quality artefacts

You essentially have several imperfect observers of intended behaviour:



None of them individually is necessarily correct.

That's important.

Code tells you:

what the system currently does.

Tests tell you:

what somebody thought should remain true.

Stories tell you:

what somebody requested at some historical point.

Production logs tell you:

what users actually do.

Confluence may tell you:

what somebody documented.

Your knowledge system should reconcile these sources rather than declaring any one of them universally authoritative.

7. That leads to a concept I'd explicitly introduce: Knowledge Status

I'd give every reconstructed requirement or behaviour a lifecycle.

For example:

```
DISCOVERED
  ↓
INFERRED
  ↓
REVIEWED
  ↓
APPROVED
  ↓
GOLD
```



Maybe:

```
candidate
review_required
approved
superseded
rejected
```



Example:

```
</> YAML

concept: Four-eye approval

status: candidate

evidence:
  - code: ApprovalService.java
  - tests: ST-231, REG-882
  - story: US-1082

inferred_rule:
  "Settlements above $1m require approval
  by a second authorised person."
```



confidence: 0.91

Human reviews it and says:

Correct, except threshold is based on currency equivalent.

Now the resulting rule becomes:

status: gold

That **gold promotion workflow** is exactly the right way to use AI here.

8. I would not let AI directly “rewrite history”

This is one place I'd be stricter than a typical LLM Wiki implementation.

Suppose AI reads:

code
tests
old stories

and derives:

“Settlement approval requires two approvers.”

Don't immediately insert that as an accepted requirement.

Instead:

AI inference
↓
candidate knowledge
↓
human review
↓
gold knowledge

The original evidence stays immutable.

So you'd maintain something like:

RAW LAYER
—
story
code
test
pipeline
Confluence

DERIVED LAYER
—
AI concepts

AI requirements
AI architecture interpretation

GOLD LAYER

approved requirements
approved ACs
approved architecture
approved business rules

That separation will save you many governance problems later.

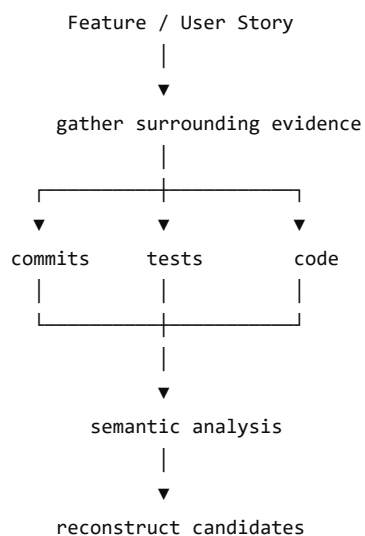
9. Your “missing documentation reconstruction” workflow is very plausible

What you described:

Existing Confluence and Azure DevOps stories don't contain enough AC/business requirement/architecture. Read code and suggest them, human reviews them, promote them to gold.

I'd implement it as a structured pipeline.

For every Feature/User Story:



Candidate outputs:

Business Requirement
Acceptance Criteria
Business Rules
Architecture Decision
Affected Components
Dependencies
Test Coverage
Known Gaps

That's quite powerful.

10. But I'd reconstruct at Feature/domain level first, not blindly at story level

This is one change I'd make to your idea.

Historical stories can be very noisy.

Suppose you had:

```
US-101
US-118
US-232
US-410
```



all modifying the same workflow over four years.

If AI reconstructs documentation independently for each story, you may get:

```
hundreds of fragmented pseudo-requirements
```



Instead:

```
stories
commits
tests
code
    ↓
cluster around capability
    ↓
Business Capability
    ↓
Current Behaviour
```



Example:

```
Capability:
Settlement Approval
```



```
Current rules:
- threshold approval
- privileged override
- escalation
- cancellation
```

```
Evidence:
US-101
US-118
US-410
ApprovalService
test_approval_*
```

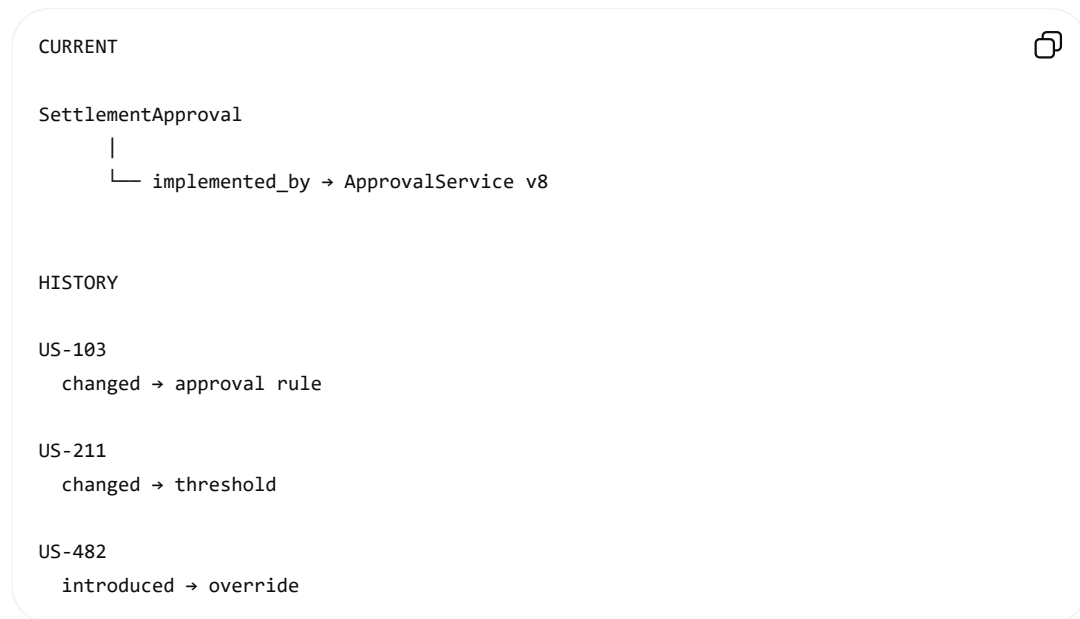
Then stories become provenance/history.

That's much cleaner.

11. Separate current truth from historical intent

This will matter enormously.

The graph should distinguish:



So an AI can answer:

Why does this logic exist?

by walking historical edges.

But when a developer asks:

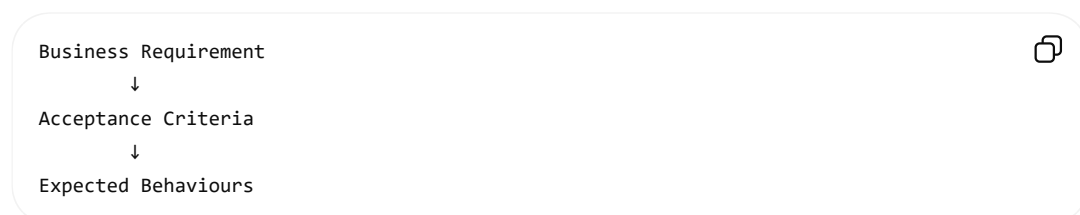
What is the current approval behaviour?

it traverses the current/gold graph.

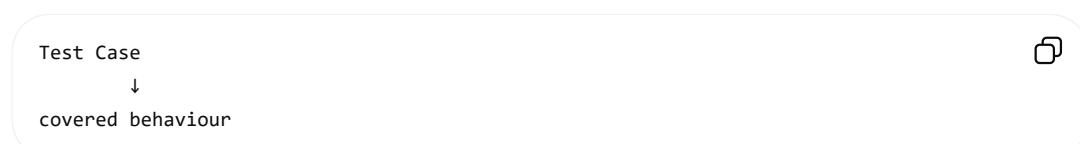
This distinction prevents old stories from poisoning current context.

12. Then your missing-test detection becomes much more reliable

Once you have Gold ACs:



and tests:



you can compare:

Expected Behaviours
MINUS
Tested Behaviours



For example:

AC01 → TC12 ✓
AC02 → TC14 ✓
AC03 → ??? ✗
AC04 → TC81 ✓



Now your AI says:

AC03 has no observed test coverage.

Azure DevOps itself supports requirement-test traceability and can show requirements that don't have associated tests, so your knowledge layer can combine that explicit traceability with semantic coverage inferred from test content.  Microsoft Learn +1

That's much better than asking an LLM:

"Do we have enough tests?"

13. And you can detect the opposite problem too

This one is potentially even more interesting.

Suppose:

TC-182:
"Senior analyst can approve settlement"



but Gold AC says:

Only manager can approve settlement.



You now detect:

TEST ↔ REQUIREMENT CONTRADICTION



Or:

code allows senior analyst
test expects senior analyst
gold requirement says manager only



Now the system tells you:

Code/Test agree.



Documentation disagrees.

That is extraordinarily valuable.

It changes the system from:

documentation search

into:

continuous knowledge consistency checking

14. You can define a consistency matrix

Something like:

Requirement	Code	Test	Gold doc	Assessment
R1	✓	✓	✓	consistent
R2	✓	✗	✓	missing test
R3	✗	✓	✓	implementation regression?
R4	✓	✓	✗	undocumented behaviour
R5	✓	✓	conflicting	investigate

This is where your idea becomes significantly more valuable than either LLM Wiki or OpenKB by themselves.

15. Pipeline traceability becomes another deterministic input

Your Azure DevOps assumption is broadly correct, with an important qualification:

Azure DevOps can maintain traceability among **work items, Git activity, builds/releases and tests**, but it is not magic; the relationships need to be created via work-item association, PR/commit linkage, test-case linkage, pipeline test publication, etc. Azure's own model exposes branches, commits, PRs and builds through the Development control of work items, and Test Plans can link automated/manual tests to requirements and pipelines.  Microsoft Learn +1

So you can derive:

US-182

|



```
├─ PR-441
│   │
│   └─ commit A
│       └─ commit B
│
├─ Build 8321
│
├─ Release 438
│
└─ TestRun 2881
```

That's excellent deterministic graph material.

16. Then add static code analysis underneath

Suppose PR-441 modifies:

ApprovalService.java



Static analysis tells you:

```
ApprovalService
  ↓ calls
AuthorizationService
```

```
ApprovalService
  ↓ writes
settlement_approval
```

```
ApprovalService
  ↓ exposes through
ApprovalController
```



Now impact analysis becomes:

```
User Story
  ↓
commit
  ↓
code files
  ↓
code dependencies
  ↓
services/APIs/tables
  ↓
business capabilities
  ↓
tests
```



That's the cross-repo impact graph you originally wanted.

You absolutely should retain that.

17. So I would use LLM Wiki/OpenKB *after* deterministic knowledge extraction

This is the conceptual sequence I'd recommend:

Stage 1

SYSTEM FACTS

Git
ADO
Pipeline
AST
Tests

↓

Stage 2

DETERMINISTIC GRAPH

code dependencies
change lineage
pipeline lineage
test lineage

↓

Stage 3

LLM SEMANTIC COMPILATION

concepts
business behaviour
requirements
architecture
relationships

↓

Stage 4

HUMAN REVIEW

candidate → gold

↓

Stage 5

CONTINUOUS ASSURANCE

missing tests
missing docs
contradictions
impact analysis



That's much safer than asking OpenKB/LLM Wiki to ingest everything and infer all relationships itself.

18. OpenKB and LLM Wiki can absolutely ingest code, but I wouldn't rely on them as code intelligence engines

This is a subtle distinction.

They can ingest:

```
.java  
.tsx  
.py  
.sql  
.md  
yaml  
etc.
```



and LLMs can reason over it.

But for things such as:

```
A calls B  
class X inherits Y  
module X imports module Y  
table A feeds table B  
API X has consumers Y/Z
```



I would prefer deterministic tooling.

LLM inference should answer:

What does this dependency *mean*?

rather than:

Does this dependency exist?

For example:

```
AST:  
ApprovalController → ApprovalService
```



```
LLM:  
This appears to implement the human approval step  
in settlement processing.
```

Perfect separation of responsibilities.

19. I would add another relationship dimension: temporal validity

Every piece of knowledge should ideally carry:

```
valid_from  
valid_to
```



version

because software behaviour changes.

Otherwise AI may combine:

2019 requirement
+
2022 test
+
2026 code



and infer nonsense.

Your graph might say:

Rule:
ManagerApprovalRequired

valid_from: release-2024.09
valid_to: current



This gives you:

"What was the expected behaviour in release 5.2?"

vs

"What is expected now?"

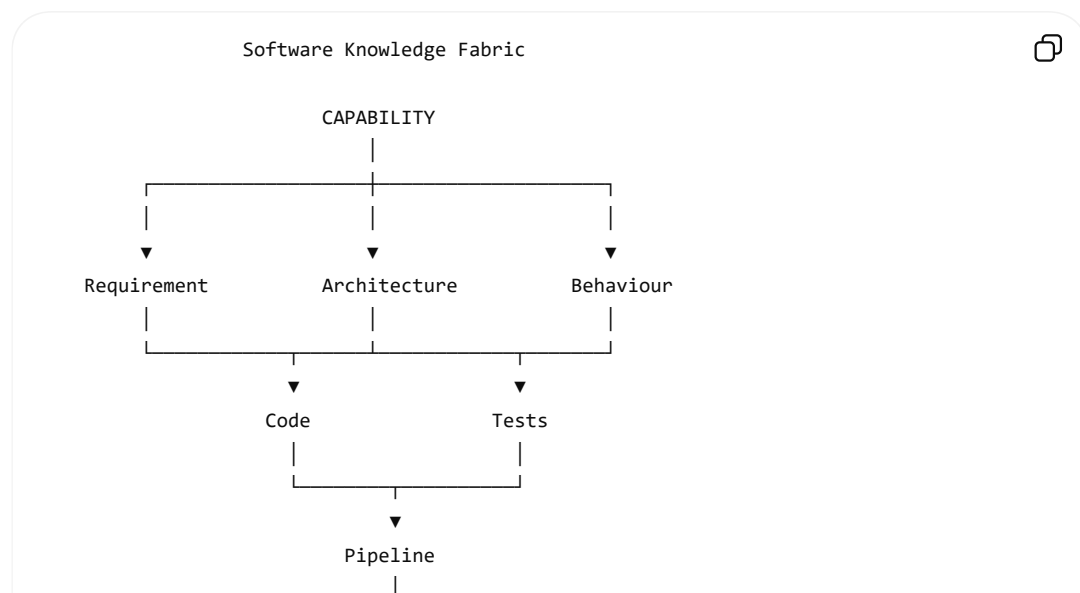


Very useful for regression investigation.

20. I'd ultimately call this a Software Knowledge Fabric, not a wiki

The system you've now described goes considerably beyond a wiki.

It's really:



▼
Runtime

With three fundamental guarantees:

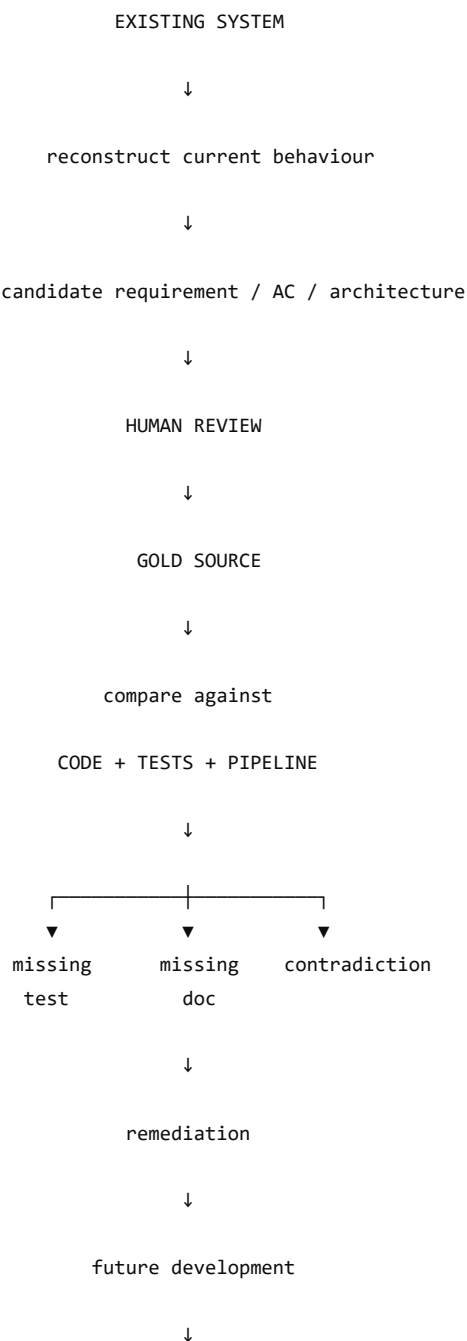
1. every relationship has evidence
2. inferred knowledge is distinguishable from deterministic knowledge
3. gold knowledge requires governance

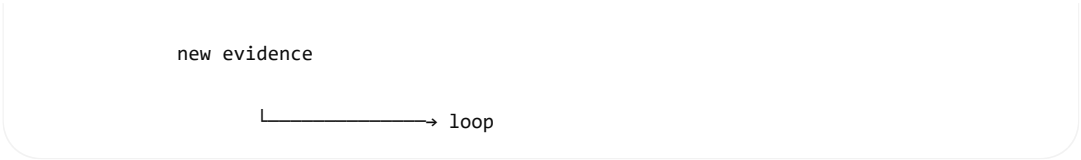


That is the architecture I'd pursue.

And your proposed workflow becomes a closed loop

I would make the operating model:





That closed loop is, in my view, **more important than choosing LLM Wiki versus OpenKB**.

Those two repos can supply parts of the implementation. Your real differentiator would be the **evidence-aware, human-governed, continuously reconciled knowledge model connecting requirement → architecture → code → tests → pipeline → runtime**.

If you build it this way, it becomes useful not only for coding agents, but also for **change-impact analysis, regression planning, release assurance, architecture review, onboarding, auditability and eventually AI-driven SDLC governance**.

   ...  Sources

Yes. Your test-case tagging approach makes the traceability much more realistic: you are deliberately creating **deterministic semantic anchors** such as `User Story` , `Feature` , and business/service function, rather than expecting Azure DevOps to infer them.

For the static-code-analysis gap, there are a few open-source options, but I would **not start with a heavyweight enterprise code intelligence platform**. For your Java + React/TypeScript environment, I would shortlist **Joern, ArchiCore, and possibly a newer multi-repo graph tool such as Grafel**.

Static analysis options

Tool	What it gives you	Java	TS/JS	Cross-repo	Weight	
Joern	Code Property Graph: AST, calls, control/data flow	✓	✓	You build aggregation	Medium	I f
ArchiCore	AST dependency graph, impact analysis, GraphML/JSON	✓	✓	Possible	Light	I I
Grafel	Multi-repo call graph, HTTP routes, message buses	Appears yes	Appears yes	✓ native	Light/medium	✓ i t

Tool	What it gives you	Java	TS/JS	Cross-repo	Weight	
Codesight	Symbols/imports/call graph in VS Code	✓	✓	Limited	Very light	
Stratify	Dependency graph using Joern/LSP/SCIP	✓	✓	Depends	Light	
Semgrep	Rules, patterns, dataflow/security	✓	✓	Not primarily graph-oriented	Light	

Joern is mature enough to deserve serious consideration. It builds a **Code Property Graph**, combining representations such as syntax, calls, control flow and data flow, and currently supports Java, JavaScript/TypeScript, Python, Kotlin and several other languages. It is Apache-2.0 licensed. [GitHub](#) +2

However, if your goal is initially:

“Give me classes/functions/services/dependencies and export the graph”

rather than vulnerability research or sophisticated data-flow analysis, **Joern may initially be more machinery than you need.**

ArchiCore is interesting because it explicitly provides AST-based dependency analysis, transitive impact analysis and export formats including JSON, CSV and GraphML, while supporting Java, TypeScript/JavaScript and many other languages.

[GitHub](#)

So for a first PoC I would probably try:

ArchiCore → prove the graph model → Joern if deeper analysis becomes necessary.

One project I'd investigate particularly carefully: Grafel

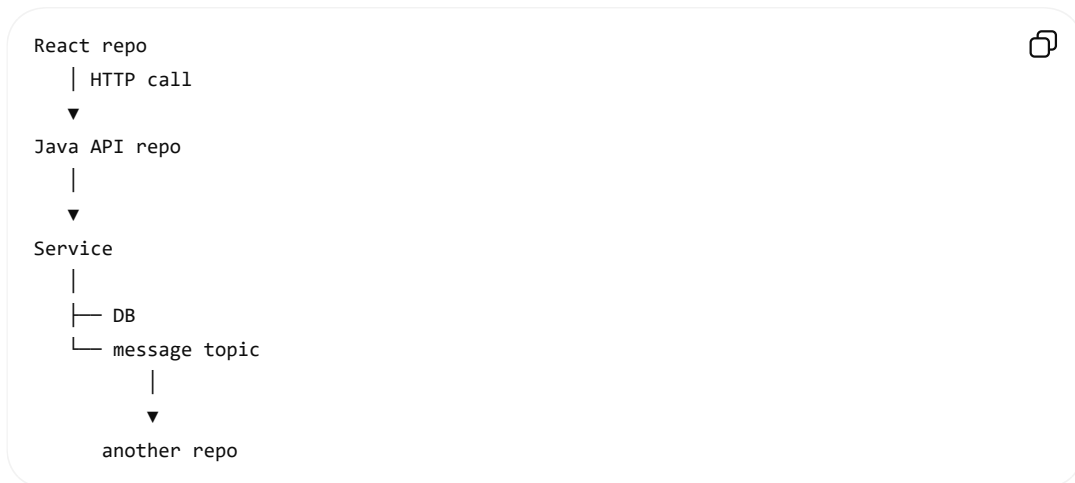
Grafel is almost suspiciously close to the problem you've described.

Its current project description includes:

- multi-repo code knowledge graphs
- cross-repo linking
- call graph navigation
- HTTP endpoint mapping
- process-flow tracing
- message-bus topology
- cross-repo dependency graphs

- confidence-scored cross-repository edges.  GitHub

That maps extremely well onto something like:



This is much closer to your desired **cross-repository impact graph** than plain AST analysis.

I would investigate it, but because it appears relatively new, I wouldn't make it a critical enterprise dependency until you've evaluated maintainability, parsing accuracy and community maturity.

But I think you're missing a few components beyond static analysis

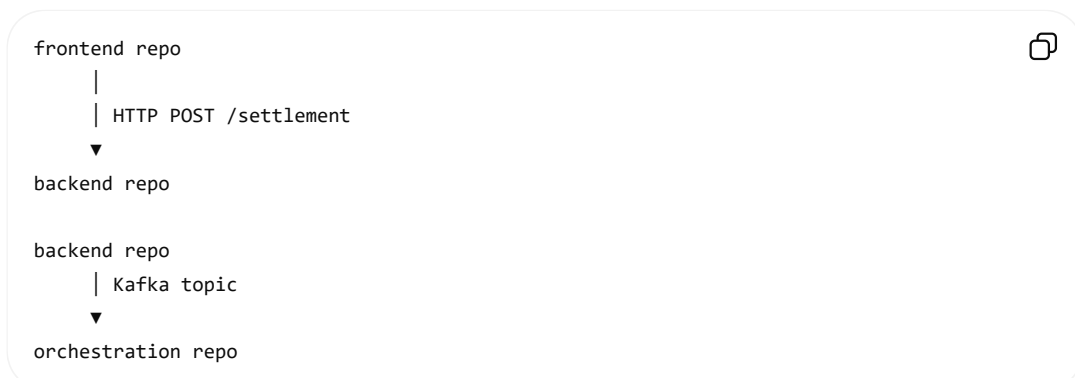
Your architecture is now quite complete conceptually, but I see roughly **six remaining technical problems**.

The first three are particularly important.

1. Cross-repo resolution

Static analysis inside one repository is comparatively easy.

The hard case is:



AST analysis alone often can't establish this relationship.

You therefore need **cross-boundary resolvers**.

For example:

```
React API client
  |
  | endpoint=/api/settlements
  |
  ▼
OpenAPI definition
  |
  ▼
Spring Controller
```



This relationship may be inferred deterministically through:

```
URL
HTTP method
OpenAPI operationId
client generator metadata
```



Likewise:

```
Producer
  |
  | topic = settlement.approved
  |
  ▼
Kafka
  |
  ▼
Consumer
```



The topic name becomes the deterministic join key.

So I'd define an explicit **integration graph layer**:

```
Code graph
  |
  ├── HTTP
  ├── event/message
  ├── database
  ├── file/storage
  └── library/package
```



That is more useful for impact analysis than method-level call graphs alone.

2. Don't let the code graph become too granular

This is a major risk.

Joern can theoretically generate enormous graphs:

```
repository
  ↓
files
  ↓
classes
  ↓
methods
  ↓
```



```
variables
↓
statements
↓
AST nodes
```

You probably **do not want all of that in your enterprise knowledge graph**.

You'll create millions of nodes very quickly.

Instead keep the detailed Code Property Graph separately:

```
Detailed Code Graph
Joern / analysis store
```



and publish selected entities into your knowledge layer:

```
Knowledge Graph
|
├─ repo
├─ service
├─ module
├─ class
├─ important method
├─ API
├─ database/table
├─ event
└─ business function
```



Then preserve references back into the lower-level graph.

Conceptually:

```
Knowledge graph:

SettlementService
  |
  └─ code_entity_id
      ▼
      Joern CPG node 8726182
```



This gives you both abstraction and drill-down.

3. You need an explicit ontology/schema

This may actually be more important than static analysis.

Before implementing the graph, define what kinds of nodes and relationships you support.

For example:

```
BUSINESS

Capability
BusinessFunction
```



BusinessRule
Requirement
AcceptanceCriterion

DELIVERY

Feature
UserStory
Bug
PullRequest
Commit
Release

SOFTWARE

System
Service
Repository
Module
API
Event
Database
Table

TEST

TestCase
TestSuite
TestExecution
TestResult

KNOWLEDGE

Document
ArchitectureDecision
CandidateKnowledge
GoldKnowledge

Then relationships:

Capability
CONTAINS → BusinessFunction

Feature
IMPLEMENTS → Capability

UserStory
IMPLEMENTS → BusinessFunction

AcceptanceCriterion
SPECIFIES → Behaviour

Commit
IMPLEMENTS → UserStory

CodeComponent
IMPLEMENTS → BusinessFunction



TestCase
VERIFIES → AcceptanceCriterion

TestExecution
EXECUTES → TestCase

Release
CONTAINS → Commit

Service
CALLS → Service

Service
PUBLISHES → Event

Service
READS_FROM → Table

Without a controlled ontology, the LLM will gradually produce:

supports
implements
realizes
provides
used_for
related_to
associated_with



all meaning approximately the same thing.

Your graph eventually becomes unusable.

4. You need identity resolution

This is less exciting but absolutely essential.

Imagine these all refer to the same thing:

Settlement Approval

settlement-approval

SettlementApprovalService

SA approval

Four-eye settlement check



LLM Wiki/OpenKB may generate separate concepts.

Your code uses another name.

Azure DevOps tags use another.

Tests may say:

```
service_function = SETTLEMENT_AUTH
```



You therefore need a canonical entity:

```
</> YAML
```



```
id: business-function:settlement-approval

name: Settlement Approval

aliases:
  - SettlementApproval
  - SA approval
  - settlement authorization
```

Everything links to that canonical ID.

Otherwise your knowledge graph fragments very rapidly.

5. You need change/version awareness

I'd make the graph **commit-aware**, rather than merely continuously replacing its state.

Suppose:

```
Release 4.7
ApprovalService → ManagerApproval

Release 5.0
ApprovalService → RiskBasedApproval
```



You want to answer both:

What's the current implementation?

and:

What changed between 4.7 and 5.0?

So static extraction should ideally produce:

```
snapshot:
  repo
  branch
  commit_sha
  timestamp
```



Some current multi-repo graph tools explicitly support graph-state comparison between indexed refs, which demonstrates the usefulness of this model.  [GitHub](#)

I would avoid storing a full duplicated graph per commit, though.

Something like:

```
current graph
+
change events
+
release snapshots
```



is probably sufficient.

6. You'll eventually want runtime evidence too

I wouldn't put this in your first implementation, but leave room for it.

Static code says:

```
A can call B
```



Runtime telemetry may say:

```
A actually calls B 180,000 times/day
```



or:

```
this path hasn't executed in six months
```



If you're using distributed tracing such as OpenTelemetry, you could eventually derive:

```
ServiceA
  OBSERVED_CALL → ServiceB

API
  OBSERVED_USE → UI workflow
```



Then you have another evidence class:

```
static relationship
vs
runtime-observed relationship
```



This helps enormously with real impact analysis.

For example:

```
static analysis:
A → B → C → D

runtime:
A → B → C

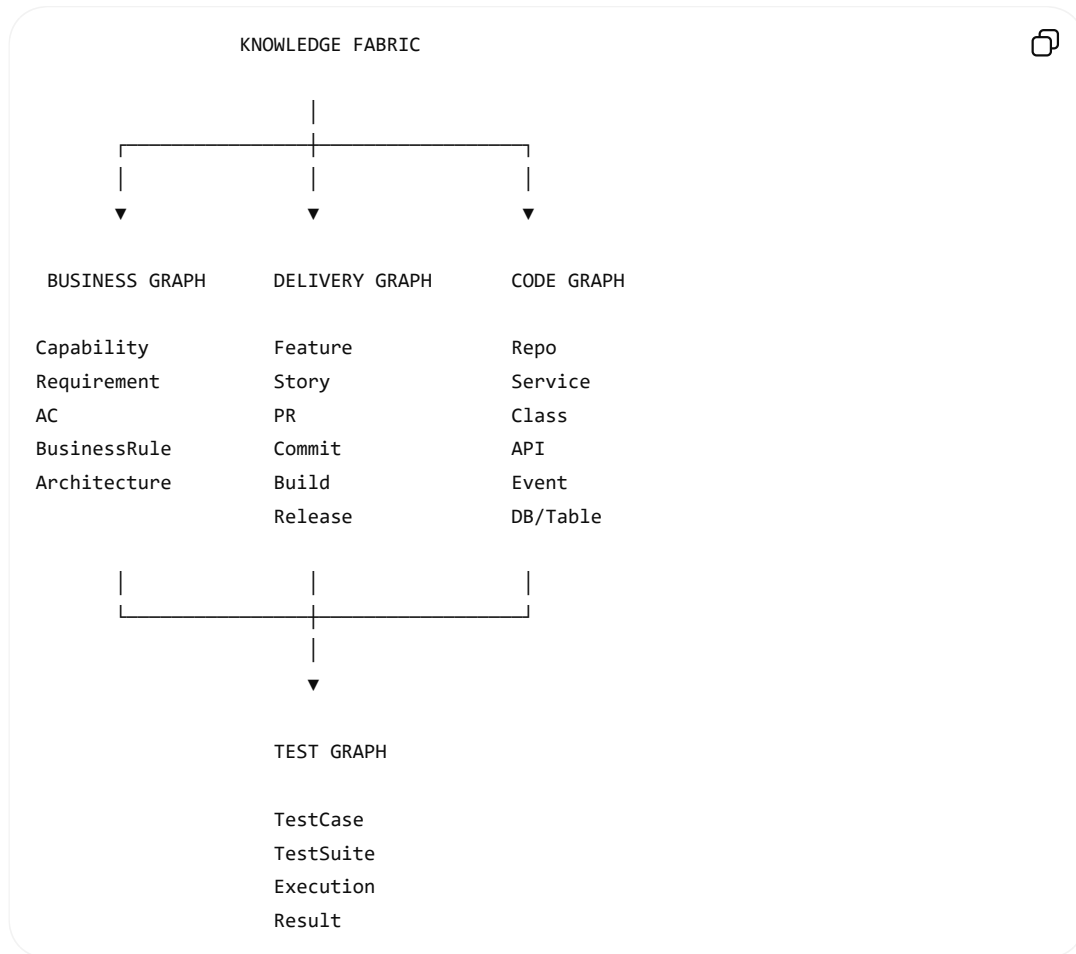
D never observed
```



You shouldn't remove the static dependency, but you now understand its operational relevance.

So I would slightly redesign the solution into four graphs

Not literally four separate graph databases—four **logical layers**.



But crucially:



That's the real knowledge graph.

Tool placement becomes clearer

I would use each technology for a narrow responsibility:

Azure DevOps APIs



delivery + pipeline + test relationships

ArchiCore / Joern / Grafel



deterministic code relationships

OpenKB



knowledge compilation
long-document reasoning
agent-facing knowledge

LLM Wiki ideas



graph expansion
review workflow
purpose
knowledge gap detection

LLM



semantic relationships
candidate BR / AC / architecture

Human



Gold promotion



This division is quite clean.

An especially important rule: deterministic evidence always wins

I would encode this into the platform.

Suppose AI says:

Service A calls Service B
confidence = 0.96



but static analysis shows no path.

Don't simply trust AI.

Store:

```
AI hypothesis:  
A → B
```



```
static evidence:  
none
```

Maybe AI inferred it from business semantics.

Then it stays:

```
relationship_type = inferred
```



If static/runtime analysis later finds:

```
A → API-X → B
```



then promote the edge evidence:

```
relationship:  
A → B  
  
evidence:  
- AI inferred  
- API contract  
- runtime trace
```



That's a much more powerful model.

What I'd use for your first PoC

I would resist implementing the entire architecture.

Pick **one service function**—for example:

```
Settlement Approval
```



and perhaps 2–3 repositories.

Then ingest:

```
Azure DevOps  
├─ 1 feature  
├─ ~10 stories  
├─ associated PRs/commits  
└─ release pipeline  
  
Test repo  
├─ system tests  
└─ regression tests  
  
Code  
├─ frontend repo  
├─ backend repo  
└─ workflow/orchestration repo
```



Confluence
└ existing relevant docs

Use:

ArchiCore or Joern → code structure

plus your manually-maintained:

test → story
test → business function
story → feature



relationships.

Then ask the system to reconstruct:

Business Function
Business Requirements
Acceptance Criteria
Architecture
Implementation map
Test coverage
Knowledge gaps



and manually review the results.

If that works for one capability, then scaling is largely an engineering exercise.

If it doesn't, you'll discover whether the problem is:

ontology
identity resolution
static analysis
LLM extraction
test metadata
or source quality



before building an enterprise platform.

My current architecture preference

I would now describe the core stack as:

OpenKB-style compiled knowledge + LLM-Wiki-style provenance/review + a deterministic Code Property Graph underneath + Azure DevOps/test traceability around it.

And of the missing pieces, I'd rank priorities:

- 1. Ontology/schema**
- 2. Static + cross-repo code graph**
- 3. Entity identity/canonicalisation**
- 4. Evidence/provenance model**
- 5. Version/change model**
- LLM knowledge reconstruction
- Gold promotion workflow

8. Automated consistency/test-gap analysis
9. Runtime evidence later

So yes, static analysis is the obvious technical gap—but **the ontology and identity model are actually the things I would design first**, because they'll determine how Joern/OpenKB/ADO/test information can all join into one coherent graph.

   ...  Sources

Okay, I will try Grafel, a side question does it support Symbols? feel free to search Grafel  Web search .
Main question is can or how to link code graph produced by grafel to the overall enterprise knowledge from other source like confluence page, azure devops

Worked for 15s >

Yes — Grafel does support what you're calling **symbols**, although it uses its own entity vocabulary rather than a generic "symbol" type. Its graph has first-class entities such as `Component` for classes/structs/modules/types and `Operation` for functions/methods/handlers; every entity carries a stable ID, qualified name, source file and line range. It also models `Schema`, HTTP endpoints, queues/topics, and process flows.  GitHub

So for something like:

```
SettlementController
└─ submitSettlement(...)
    └─ SettlementService.process(...)
```

Grafel can represent the class/module and method/function level and connect them through edges such as `CALLS`, `IMPORTS`, `USES`, `EXTENDS`, and `OVERRIDES`. Statically resolved edges normally get high confidence, while heuristic/cross-repo edges are explicitly lower-confidence.  GitHub

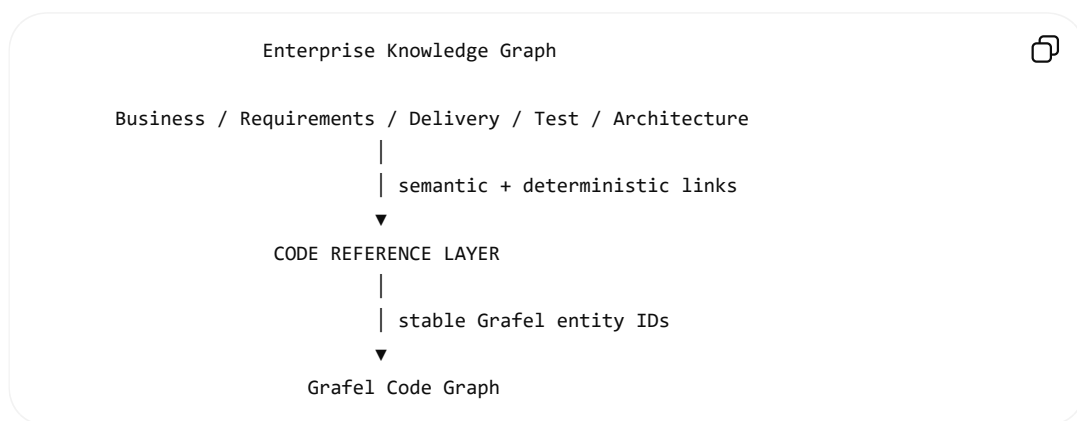
That is actually very useful for the enterprise-knowledge design, because **Grafel already gives you stable code-side identifiers to attach higher-level knowledge to.**

The main question: how should Grafel connect to Confluence + Azure DevOps?

I would **not merge the Grafel graph wholesale into the enterprise knowledge graph.**

Instead, make Grafel the authoritative **Code Graph service**, and create a higher-level enterprise graph that references selected Grafel entities.

Think:



Grafel itself remains optimised for:

```
function
method
class
module
API
message topic
DB object
call path
cross-repo dependency
```

while your enterprise graph represents:

```
Business Function
Feature
User Story
Acceptance Criterion
Architecture Component
Test Case
Release
Document
```

That separation is important.

A concrete example

Suppose Azure DevOps contains:

```
Feature F100
"Settlement Processing"
```

and:

```
User Story US-421
"Require second approval for large settlements"
```

Your test metadata says:

```
TC-882
```

```
user_story = US-421
business_function = Settlement Approval
```

Confluence has:

Settlement Processing Architecture



Grafel extracts:

```
repo: settlement-api
```



```
Component:
```

```
SettlementController
```

```
Operation:
```

```
SettlementController.submit()
```

```
Operation:
```

```
SettlementService.process()
```

```
Operation:
```

```
ApprovalService.requireApproval()
```

```
Schema:
```

```
Settlement
```

and deterministic edges:

```
SettlementController.submit
```

```
CALLS
```

```
SettlementService.process
```

```
SettlementService.process
```

```
CALLS
```

```
ApprovalService.requireApproval
```



Now your enterprise graph can say:

```
BusinessFunction:
```

```
Settlement Approval
```



```
IMPLEMENTED_BY
```

```
grafel://settlement-api/ApprovalService.requireApproval
```

And:

```
US-421
```

```
IMPLEMENTS
```

```
Settlement Approval
```



And:

```
TC-882
```

```
VERIFIES
```

```
Settlement Approval
```



You don't need to copy the whole Grafel graph.

You simply retain a **reference to Grafel's entity**.

Why I prefer references rather than one giant graph

Because your graph scales are radically different.

The enterprise graph might contain:

```
10,000 stories
3,000 test cases
500 features
1,000 requirements
200 architecture components
```



while Grafel might contain:

```
500,000 operations
100,000 components
millions of edges
```



If you pour all of that into Neo4j or another enterprise graph, then queries such as:

“What business capability is affected?”

start competing with:

“What are the 8,314 local method calls underneath this class?”

That's unnecessary.

Keep:

```
Enterprise Knowledge Graph
= semantic abstraction

Grafel
= detailed code intelligence
```



and traverse into Grafel when needed.

The bridge is an important concept

I would introduce a specific entity:

```
CodeEntityRef
```



For example:

```
</> YAML
```



```

id: code-ref:approval-service-require-approval

provider: grafel

repo: settlement-api

grafel_entity_id: <stable ID>

qualified_name:
  ApprovalService.requireApproval

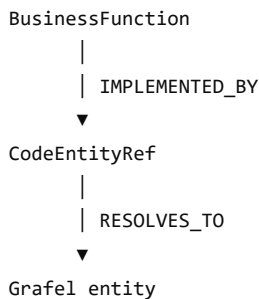
entity_kind:
  Operation

source:
  src/.../ApprovalService.java

resolution_status:
  resolved

```

Then:



That indirection may look unnecessary initially, but it becomes very valuable.

Why use a bridge entity?

Because code moves.

Today:

```
ApprovalService.requireApproval()
```

Tomorrow someone refactors it into:

```
ApprovalPolicy.evaluate()
```

The business function hasn't changed.

If your business graph directly stores:

```

Settlement Approval
  ↓
ApprovalService.requireApproval()

```

the link may disappear.

Instead:

```
Settlement Approval
  ↓
CodeEntityRef:
"implementation of Settlement Approval"
  ↓
current Grafel entity
```



Then your reconciliation job can update:

```
old symbol
  ↓ replaced_by
new symbol
```



without modifying the business knowledge itself.

That will make the architecture much more robust.

There are four good ways to create these cross-layer links

I'd classify them exactly the same way we discussed earlier.

1. Explicit metadata — highest quality

For example, developers can annotate Azure DevOps or test metadata with:

```
business-function: settlement-approval
```



and perhaps a repo/component:

```
component: settlement-api
```



That gives you:

```
US-421
  → BusinessFunction
  → repo/component
```



Then Grafel narrows the implementation mapping.

This is very strong.

2. Delivery-history linkage

Suppose:

```
US-421
  ↓
PR-902
```



↓
commit abc123

The commit modifies:

ApprovalService.java
SettlementService.java



Grafel knows the entities inside those files.

So you can derive:

US-421
CHANGED
ApprovalService.requireApproval



That's not merely AI inference.

It's a **delivery-derived relationship**.

I'd mark it:

evidence_type = delivery_trace



This is likely to be one of your best bridges.

Then Grafel makes this much stronger

Git says:

US-421
↓
ApprovalService.java



But Grafel can determine that within that file:

ApprovalService.requireApproval



was affected and that it connects to:

SettlementService.process
↓
ApprovalService.requireApproval
↓
AuthorizationService.hasRole



Grafel specifically supports caller/dependency traversal and multi-repo path tracing, including HTTP surfaces and cross-repo links. GitHub +1

So your bridge becomes:

ADO
↓



```
commit/file
↓
Grafel entity
↓
code dependency neighbourhood
```

That's excellent for impact analysis.

3. Structural anchors

For some relationships you can join through stable technical identifiers.

Examples:

```
OpenAPI operationId
HTTP route
Kafka topic
database table
package/module
service name
```



Suppose Confluence says:

```
Settlement Approval uses POST /settlement/{id}/approve
```



and Grafel has:

```
http_endpoint_definition
POST /settlement/{id}/approve
```



Then the join is near-deterministic.

Likewise:

```
Confluence:
publishes SETTLEMENT_APPROVED

Grafel:
ApprovalService
  PUBLISHES_TO
  SETTLEMENT_APPROVED
```



Grafel explicitly extracts HTTP endpoint definitions/call sites and message-bus topology.  GitHub +1

These are ideal semantic anchors.

4. AI semantic mapping

Finally you have cases where no explicit key exists.

For example a Confluence page says:

“The operator must obtain a second authorisation before settlement release.”

Grafel knows:

```
ApprovalService.requireApproval
AuthorizationService.checkSecondApprover
```



An LLM can infer:

```
Business rule
    LIKELY_IMPLEMENTED_BY
AuthorizationService.checkSecondApprover
```



But now you clearly label it:

```
evidence_type: ai_inferred
confidence: 0.86
status: candidate
```



Human review can promote it.

This is where OpenKB/LLM Wiki becomes valuable.

So the bridge-resolution hierarchy should be

I'd use:

```
1. explicit identifier
  ↓
2. delivery trace
  ↓
3. technical structural match
  ↓
4. AI semantic inference
```



The further down you go, the lower the confidence.

For example:

Mapping	Evidence	Confidence
Story → PR	ADO link	1.0
PR → commit	Git	1.0
commit → changed Grafel entity	git diff + symbol range	~0.98
API → handler	Grafel static resolution	~0.95
Business Function → API	approved metadata	1.0
Business Function → method	LLM inference	0.75–0.95

That evidence trail is exactly what you want.

Grafel's own confidence model fits your design surprisingly well

Grafel already distinguishes edge certainty.

Its docs state that statically resolved edges generally default around 0.95, while heuristic and cross-repo resolution receives lower confidence. Cross-repo relationships are stored as overlay links and also carry confidence. [GitHub](#)

It also explicitly keeps **residual edges** when it knows a relationship exists but can't confidently resolve the target, instead of pretending certainty or dropping the relationship. [GitHub](#)

I really like this philosophy for your system.

You could extend the same pattern upward:

Enterprise relationship status

RESOLVED
CANDIDATE
UNRESOLVED
REJECTED
SUPERSEDED



That means you don't force uncertain semantic mappings to become facts.

I would therefore avoid "Grafel → OpenKB import everything"

A tempting architecture would be:

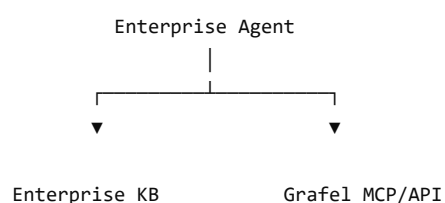
Grafel
↓ export everything
OpenKB
↓
enterprise KB

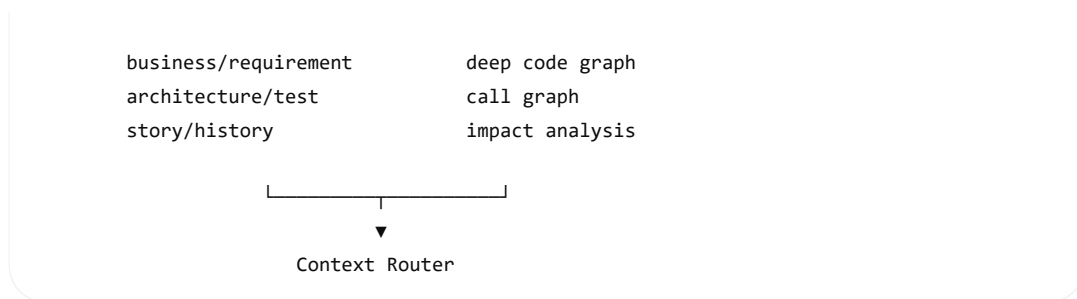


I don't think that's ideal.

OpenKB doesn't need every method call.

Instead:

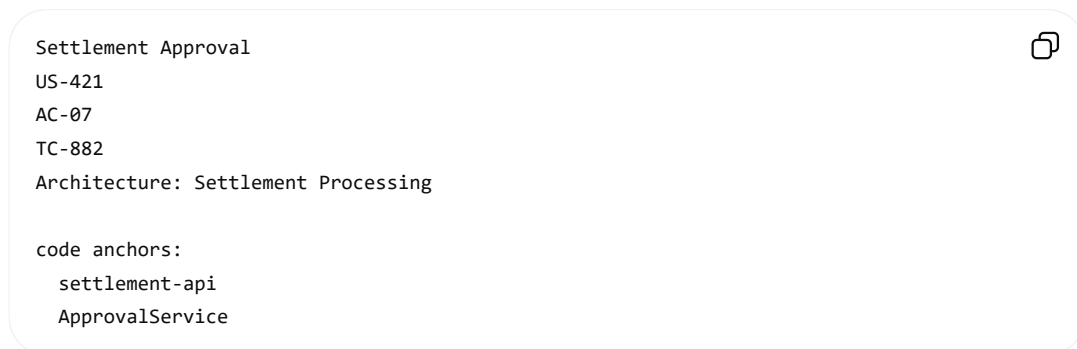




The agent asks the enterprise KB:

Which capabilities are relevant?

Then it gets:



Only then does it query Grafel:

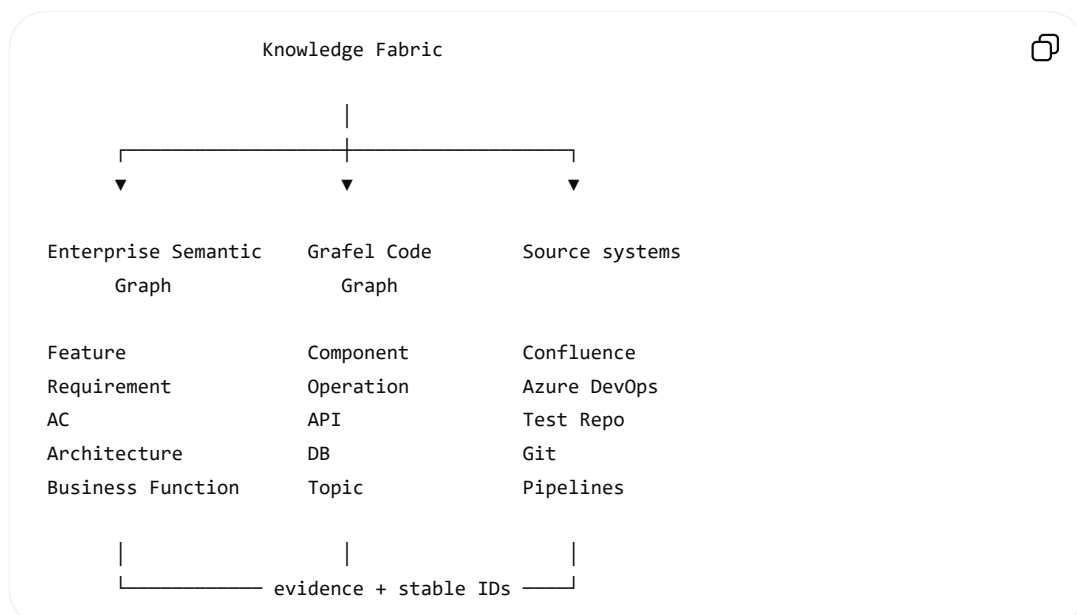
Trace downstream from `ApprovalService.requireApproval`.

This dramatically reduces context.

That means the knowledge architecture becomes federated

I think **federated knowledge** is a better description than "one giant knowledge graph."

You have:



One reasoning layer sits over them.

I'd put OpenKB here

OpenKB becomes primarily:

```
documents
  ↓
semantic extraction
  ↓
BusinessFunction
Requirement
Architecture
Behaviour
concept/entity pages
```



and perhaps stores:

```
code_refs:
- grafel://settlement-api/...
```



But it doesn't attempt to replicate Grafel's internal graph.

Likewise, LLM Wiki concepts such as source provenance and review become part of the semantic layer.

Azure DevOps becomes another authoritative graph source

Rather than "documents to ingest," I'd treat ADO partly as structured graph data.

For example:

```
Feature
├─ UserStory
│   ├── PR
│   │   └─ Commit
│   └─ TestCase
│       └─ Release
```



Don't ask the LLM to reconstruct these relationships from text.

Use ADO APIs.

Then separately send:

```
title
description
comments
attachments
```



to the LLM knowledge compiler.

This maintains the distinction between:

STRUCTURE
deterministic

CONTENT
semantic



Same principle for Confluence

A Confluence page itself gives deterministic information:

page ID
space
parent
author
version
timestamp
links



The page content gives:

business concepts
requirements
architecture
rationale



So ingest it as:

Confluence API
├── metadata/links → deterministic graph
└── page content → OpenKB/LLM extraction



Again: separate facts from interpretation.

Your complete mapping pipeline could look like this

AZURE DEVOPS

Feature → Story → PR → Commit

▼

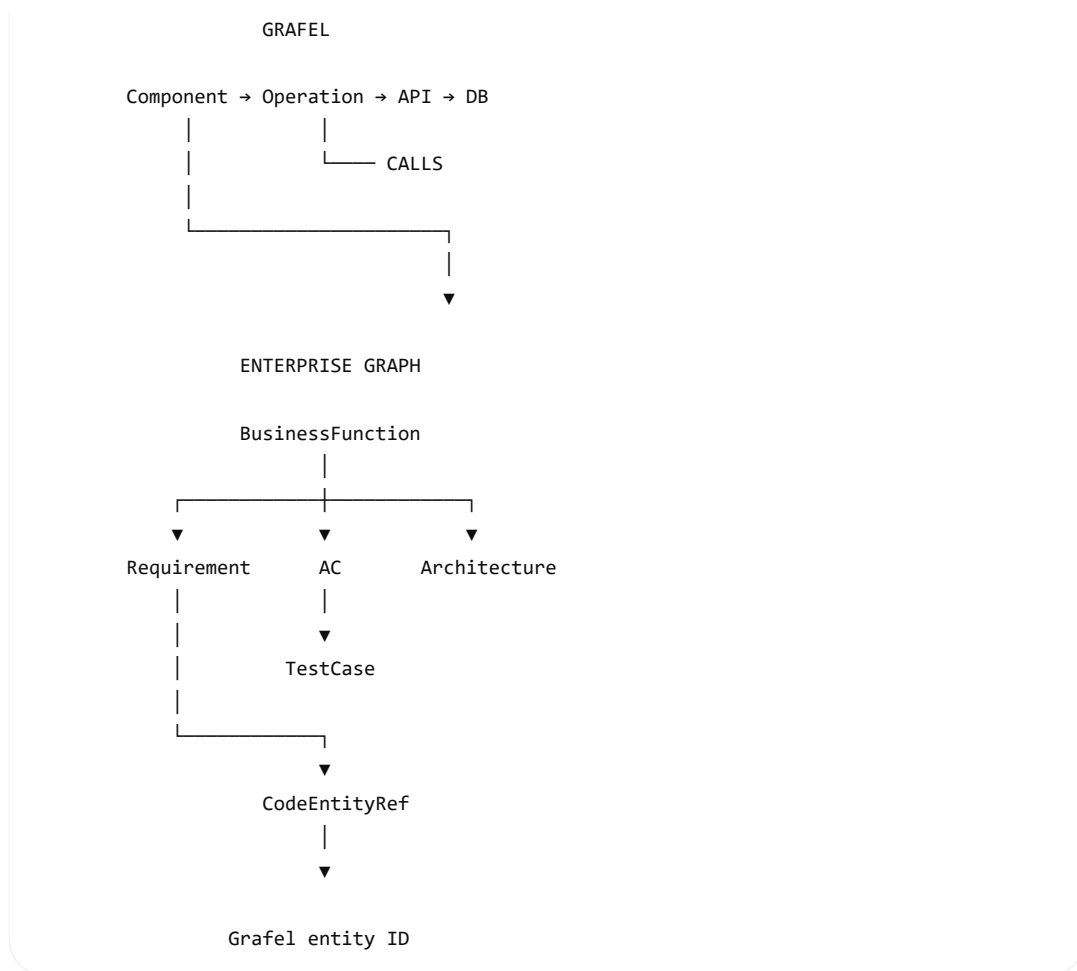
changed files

▼

symbol-range resolver

▼





That's how I would join everything.

One implementation detail I would insist on

Never use names as your primary foreign keys.

Don't do:

```
business_function:
  "settlement approval"

code:
  "ApprovalService"
```



because names change.

Use:

```
enterprise_id:
  bf-001928

source_refs:
  ado: feature/123
  confluence: page/83721
  grafel: entity/...
  git: repo+commit+path
```



Names become labels.

IDs become identity.

This will matter a lot after several years.

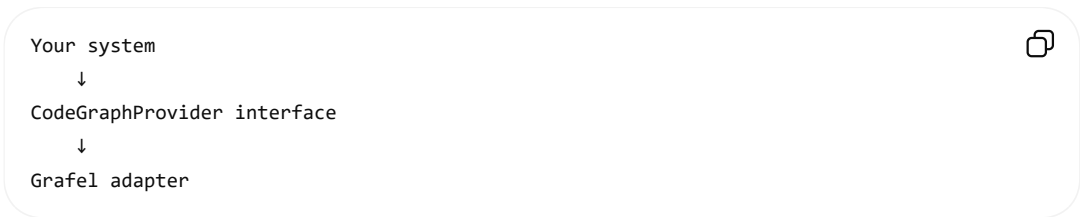
One caveat about Grafel

I should correct the level of confidence from my previous recommendation: **Grafel is technically very interesting, but it's still explicitly a v0.x project** and its graph schema/API may change between minor releases. The repo itself warns about this.

 GitHub

So for your PoC I think it's a very good candidate.

For production I would put an adapter around it:

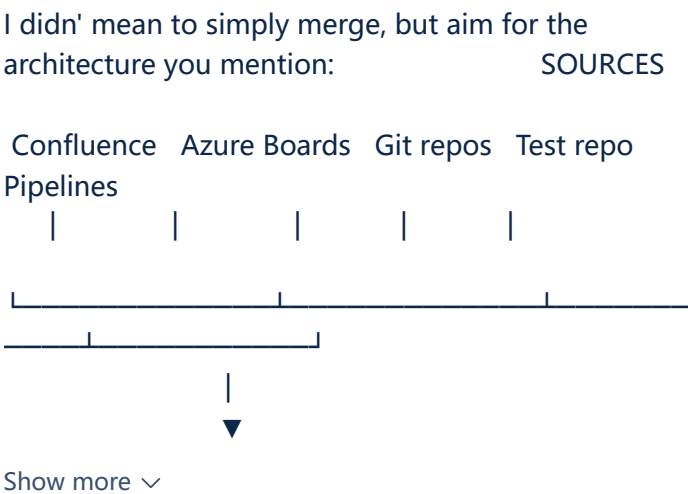


rather than letting your enterprise ontology depend directly on Grafel's internal schema.

Then if you later replace Grafel with Joern, SCIP, Sourcegraph-style indexing, or an internal analyser, the rest of the knowledge platform stays intact.

That adapter boundary may be one of the most important early engineering decisions.

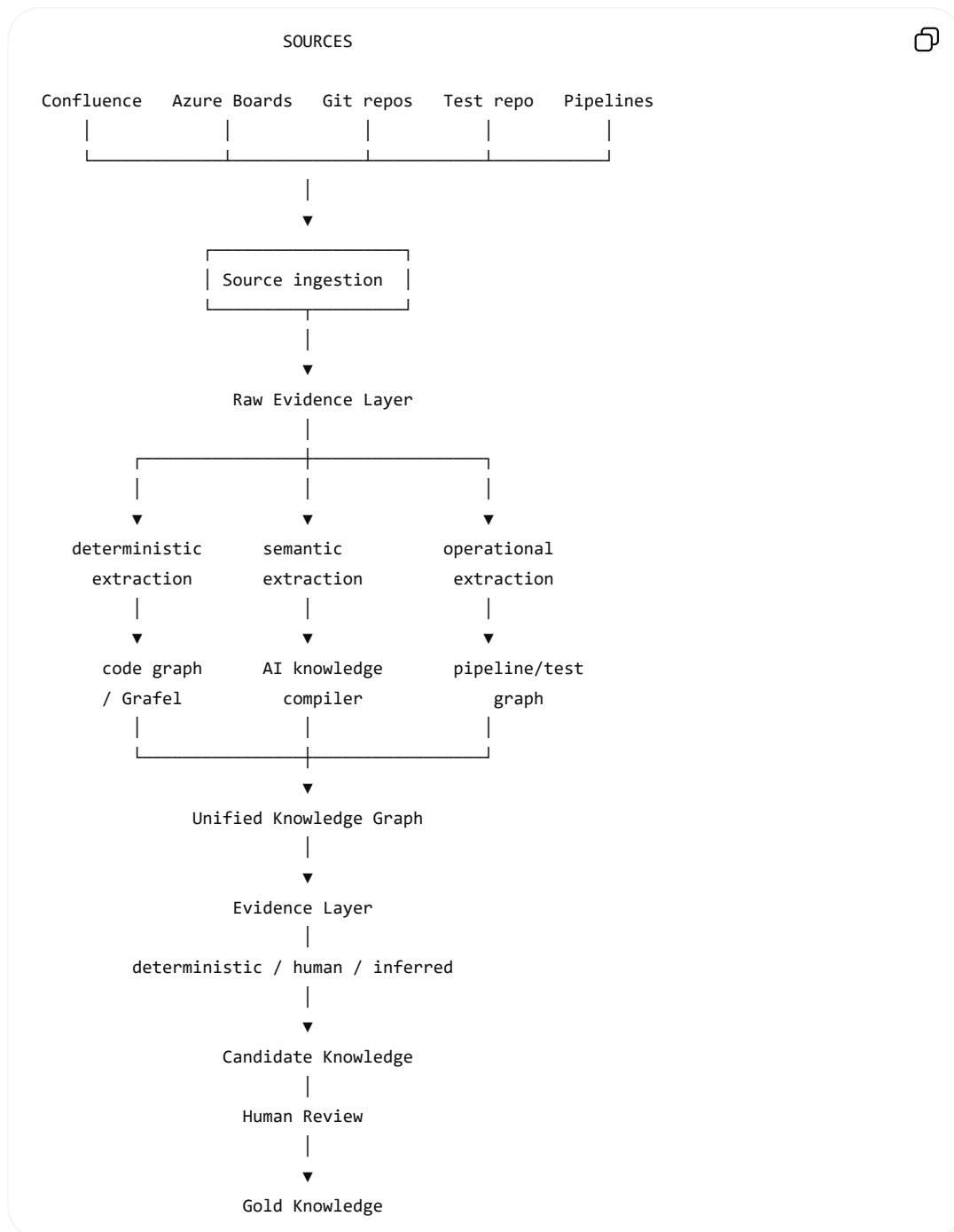
   ...  Sources



Yes — that architecture is exactly what I would recommend. The key is that **“Unified Knowledge Graph” means unified at the semantic/query layer, not necessarily physically merged into one graph engine.**

Your diagram is sound. I would only make two refinements.

First, I would separate **raw source facts** from **derived knowledge** more explicitly:



That **Raw Evidence Layer** matters because you always want to be able to answer:

“Why does the system think this?”

and trace all the way back to the original commit, test case, pipeline run, Confluence page, or ADO work item.

Second, I would not make “Gold Knowledge” a separate copy of the whole graph. I would make it a **status/governance state on selected semantic entities and edges**.

For example:

```
BusinessRule:
"Settlement above threshold requires second approval"

status = GOLD
```



while:

```
BusinessRule:
"Senior analyst can override approval"

status = CANDIDATE
evidence_type = AI_INFERRED
confidence = 0.81
```



Both can exist in the same graph.

That gives you much better lifecycle management.

Where Grafel fits

Grafel sits squarely here:

```
Git repos
|
▼
deterministic extraction
|
▼
Grafel Code Graph
```



Its responsibility is:

```
Repo
File
Component
Symbol/Operation
Call
Import
API
Event/topic
Dependency
cross-repo path
```



Then you bridge selected code entities into the unified semantic graph.

For example:

```
BusinessFunction
Settlement Approval
|
| IMPLEMENTED_BY
▼
CodeEntity
ApprovalService.requireApproval
|
| resolved through
```



▼
Grafel

The unified graph doesn't need to replicate every local variable or AST node.

OpenKB / LLM-Wiki concepts sit in the middle column

Something like:

Confluence
Azure Boards text
test descriptions
code comments
architecture docs
|
▼
semantic extraction
|
▼
Knowledge Compiler



producing:

Capability
BusinessFunction
Requirement
AcceptanceCriterion
Behaviour
BusinessRule
ArchitectureComponent
ArchitectureDecision
Concept



with source evidence attached.

That is where I would borrow OpenKB's compilation pattern and LLM Wiki's provenance/review concepts.

Azure DevOps is actually feeding two branches

Azure DevOps shouldn't be treated as a single "document source".

It contains both **structured facts** and **unstructured semantics**.

For example:

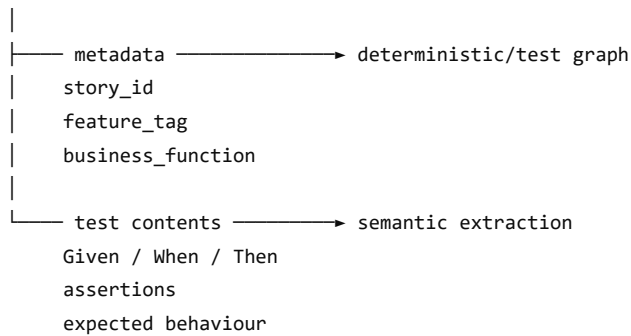
Azure Boards
|
|----- structured -----> deterministic graph
|
| Feature → Story
| Story → PR
| PR → Commit
|
|----- text -----> semantic compiler



```
title
description
comments
existing AC
```

Same for tests:

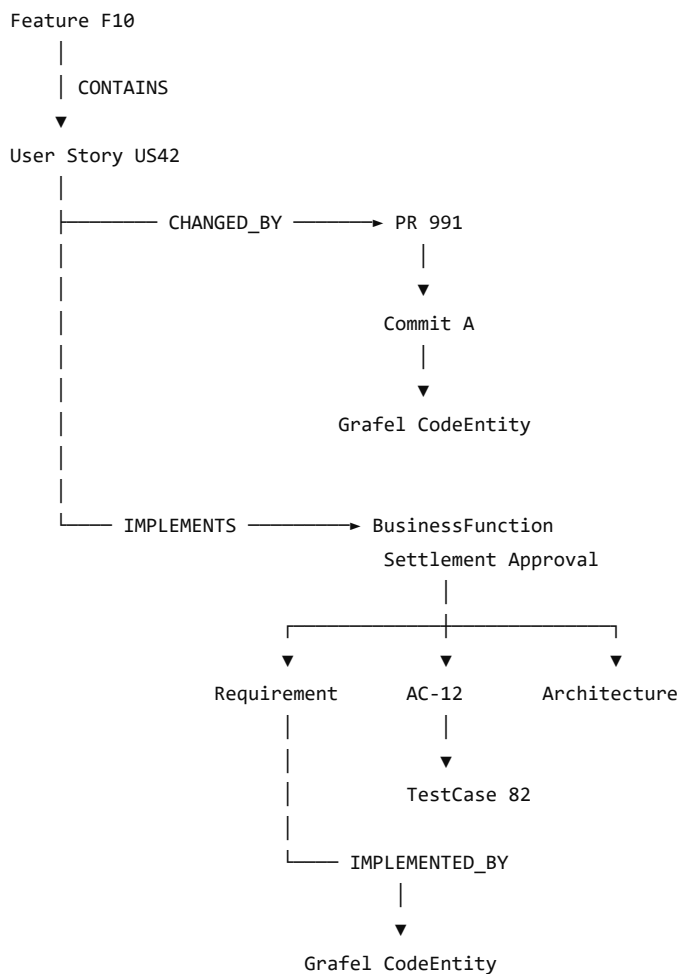
Test Repo / ADO Test Plans



This distinction is crucial.

The graph that emerges is therefore heterogeneous

A simple example:



And critically, every edge has metadata.

For example:

</> YAML



```
edge:
  from: US42
  to: PR991
  type: CHANGED_BY

evidence:
  type: deterministic
  source: azure_devops
```

versus:

</> YAML



```
edge:
  from: BF_SETTLEMENT_APPROVAL
  to: ApprovalService.requireApproval
  type: IMPLEMENTED_BY

evidence:
  type: inferred
  source:
    - code
    - test
    - story
  confidence: 0.91

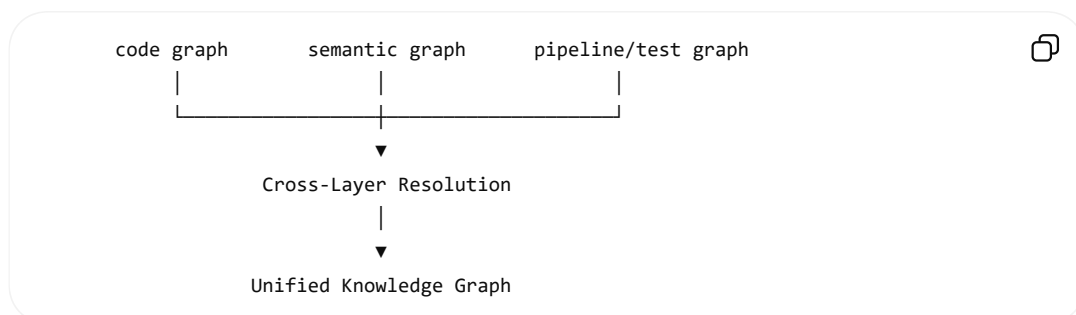
review_status: approved
```

That gives you exactly the distinction you were asking about earlier.

The really important part is the cross-layer linker

I think this is the missing architectural box in your diagram.

I'd actually insert:



Because building each graph isn't actually the hardest problem.

The harder problem is:

“How do I know that this business function corresponds to these symbols, tests, stories and architecture objects?”

That component should use several strategies:

1. explicit IDs/tags
2. ADO delivery relationships
3. technical identifiers
4. structural matching
5. semantic/LLM matching
6. human confirmation



For example:

```
TestCase 82  
tag: business-function:settlement-approval
```



gives you a deterministic edge.

But:

```
Settlement Approval  
?  
ApprovalService.validateSecondApprover()
```

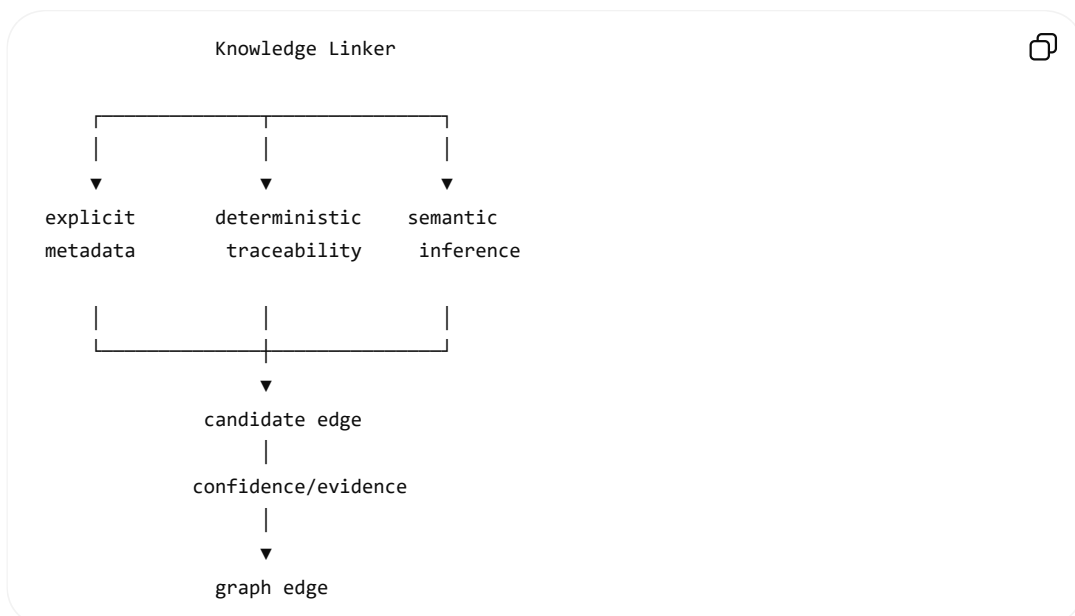


may initially be an inferred edge.

That becomes the job of the cross-layer resolution service.

I would actually call this component the “Knowledge Linker”

Its job:



Examples:

Explicit

```
TestCase.business_function_id  
=  
BF-102
```



→ confidence 1.0.

Delivery-derived

Story → PR → Commit → changed symbol



→ high-confidence relationship.

Technical

Confluence mentions POST /api/approve



Grafel contains:

POST /api/approve → ApprovalController.approve

→ near deterministic.

AI inferred

business rule text

+

method semantics



→ candidate relationship.

Your solution therefore has roughly eight main components

If you were drawing the actual target architecture, I'd use:

1. Source Connectors



2. Raw Evidence Store

3. Deterministic Extractors

- ADO
- Git
- Grafel
- test metadata

4. Semantic Knowledge Compiler

- OpenKB-like

5. Operational Extractor

- pipelines
- releases
- test execution

6. Knowledge Linker

- cross-layer resolution

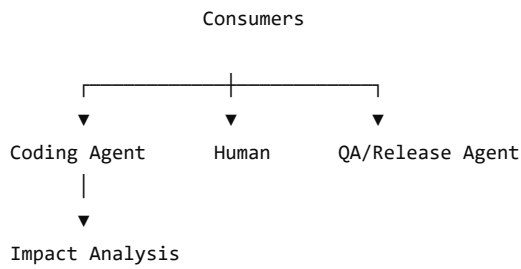
7. Unified Knowledge Graph

- ontology
- provenance
- confidence

- temporal validity

8. Governance / Gold Promotion

Then above that:



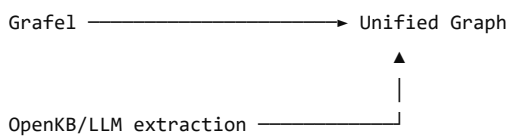
One thing I'd avoid

I wouldn't define the flow as:

Grafel → LLM → Enterprise graph

because you don't want the LLM to mediate deterministic code facts.

Instead:



The LLM may add semantic meaning to Grafel entities, but it shouldn't be required to preserve their deterministic relationships.

For example:

Grafel fact:

A CALLS B

should remain:

A CALLS B
evidence = static_analysis

not:

LLM reads A and B
→ concludes A probably calls B

That distinction is fundamental.

The architecture is now coherent enough for a PoC

At this point, I would stop adding major conceptual components and actually test the architecture on one vertical slice.

For example:

Business Function:
Settlement Approval



Sources:
- 1 feature
- 10-20 stories
- Confluence pages
- 2-3 repos
- regression/system tests
- corresponding pipelines/releases

Build:

Grafel code graph
+
ADO/test graph
+
semantic knowledge
+
Knowledge Linker
+
small unified graph



Then test whether you can reliably answer five questions:

- 1. What is Settlement Approval supposed to do?**
- 2. Which code implements it across repos?**
- 3. Which stories changed it and why?**
- 4. Which tests verify each acceptance criterion?**
- 5. If I change this component, what business functions/tests/repos may be affected?**

If those five work, you have validated the core architecture. Everything else—gold promotion, automatic document reconstruction, release assurance, runtime evidence—is an incremental extension rather than a redesign.

    Sources

ChatGPT can make mistakes. Check important info.